Enterprise System Integration (MTAT.03.229)

LECTURE 6: DOMAIN-DRIVEN DESIGN (DDD) APPROACH - II

MOHAMAD GHARIB

UNIVERSITY OF TARTU

Domain modeling - Recap

A ***domain model*** should include a set of ***fundamental language constructs*** that represent the key ***concepts*** of the domain along with the ***relationships*** among them as well as a set of ***constraints*** that govern how such constructs can be used to produce valid models*.

The development of a ***domain model*** required several ***iterations***, and came to its end when the model ***is considered complete*** with respect to the main properties of the domain that ***we aim to capture***.

* A systematic approach to domain-specific language design using UML. In: 10th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing, ISORC 2007, IEEE, pp 2–9

Recap

Customer

Product catalog

Order Request

Payment

Purchase Order

Product item

Delivery order

invoice

Orders management Employee

Inventory Employee

Recap

Customer

Product catalog

Order Request

Product item entry

Inventory

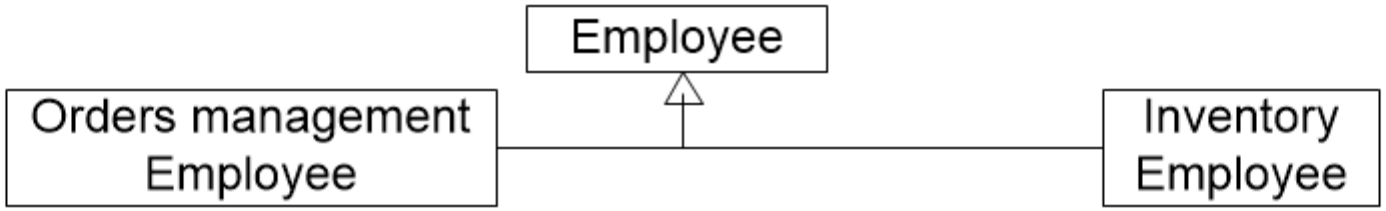
Payment

Purchase Order

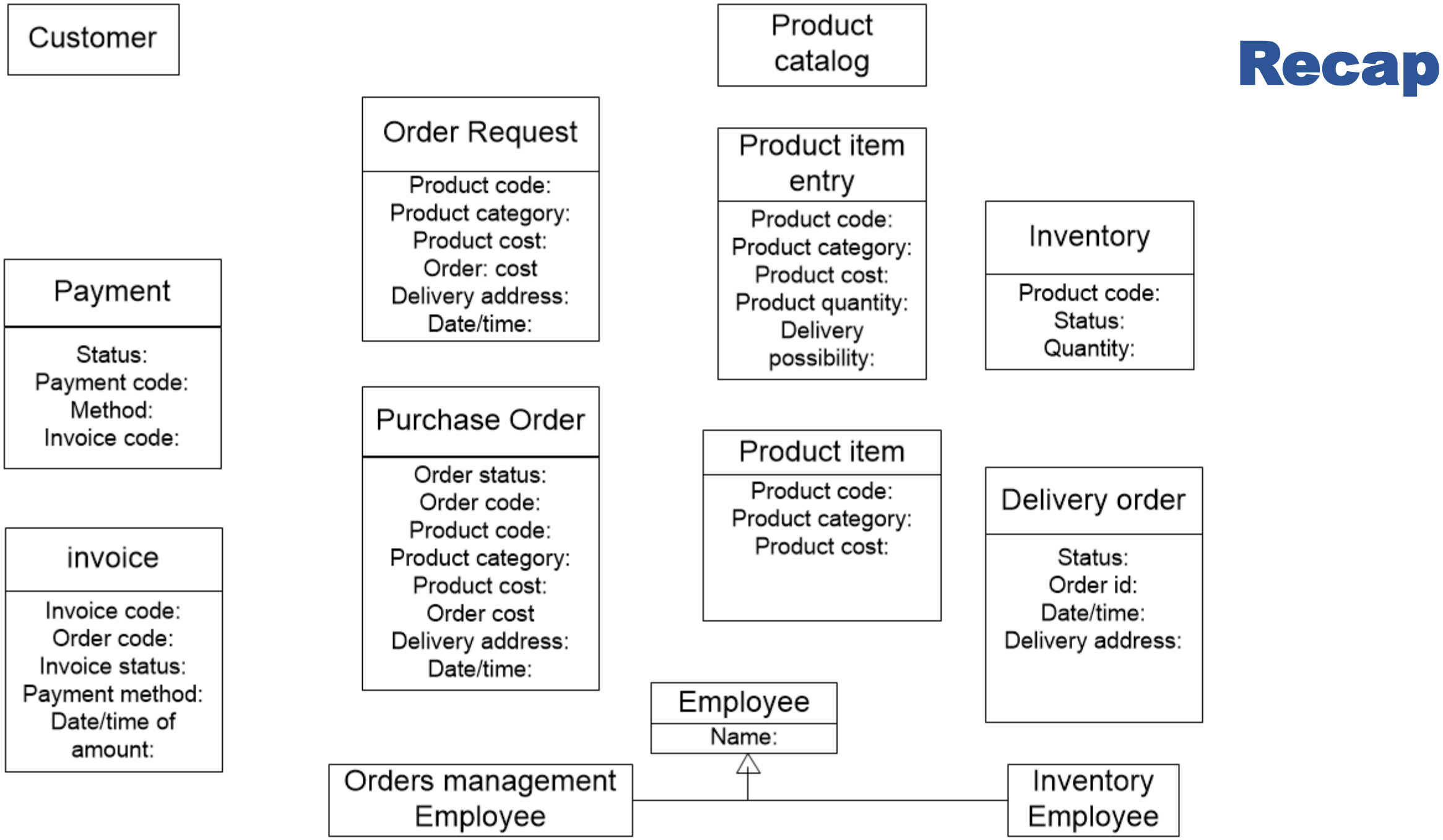
Product item

Delivery order

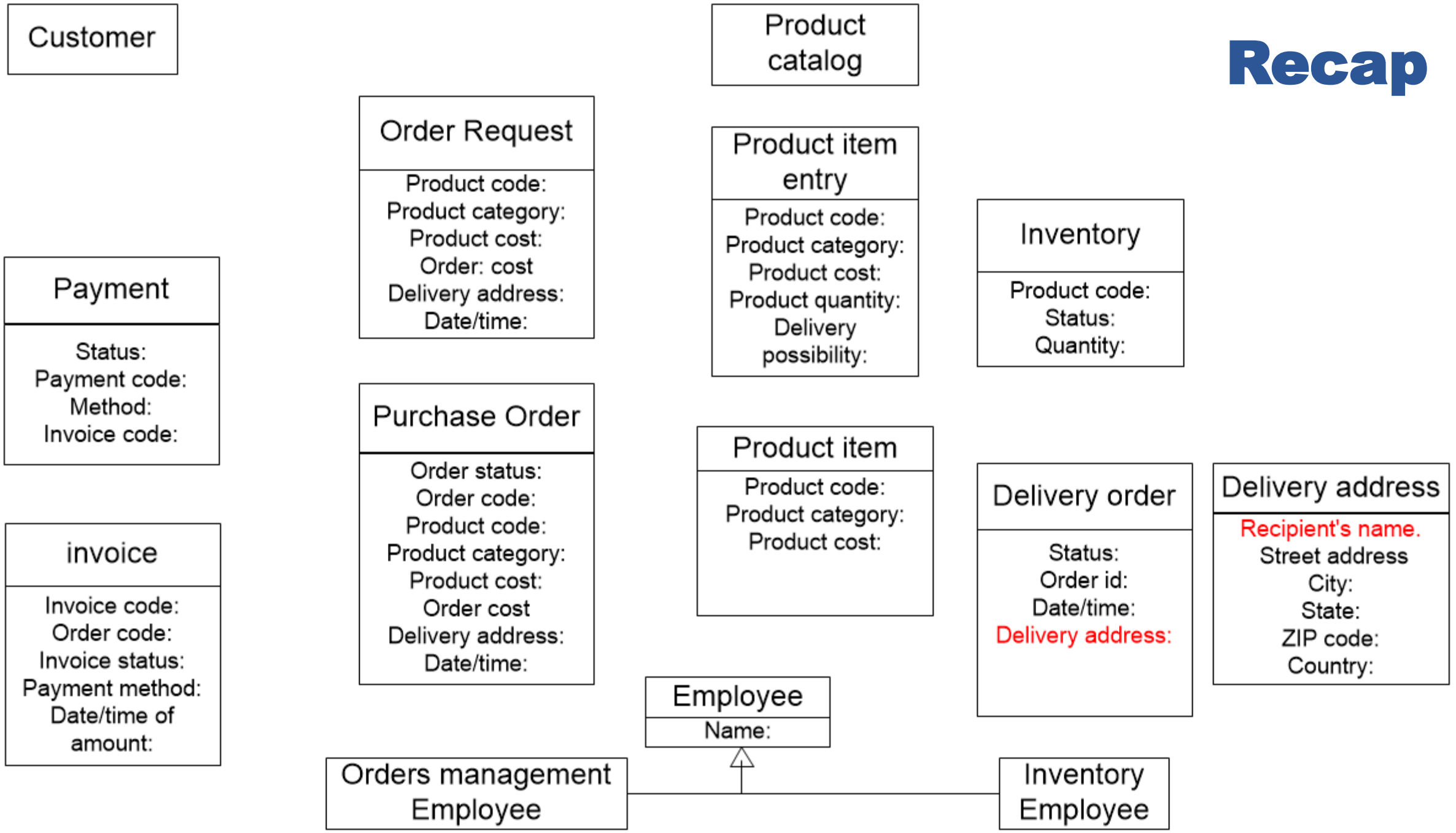
invoice



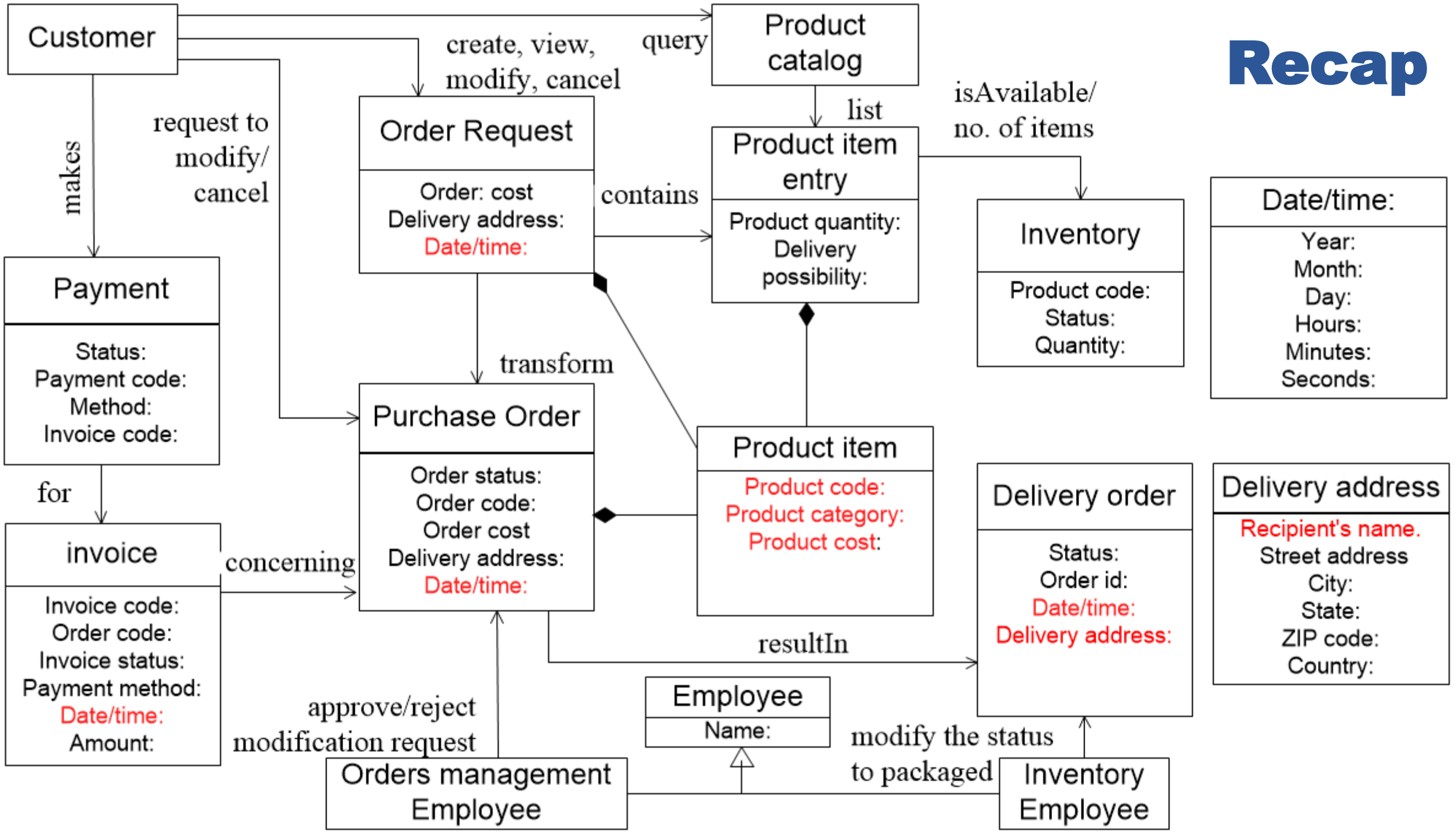
Recap


Customer
Product catalog
Order Request
Product code:
Product category:
Product cost:
Order: cost
Delivery address:
Date/time:
Product item entry
Product code:
Product category:
Product cost:
Product quantity:
Delivery possibility:
Inventory
Product code:
Status:
Quantity:
Payment
Status:
Payment code:
Method:
Invoice code:
Purchase Order
Order status:
Order code:
Product code:
Product category:
Product cost:
Order cost
Delivery address:
Date/time:
Product item
Product code:
Product category:
Product cost:
Delivery order
Status:
Order id:
Date/time:
Delivery address:
invoice
Invoice code:
Order code:
Invoice status:
Payment method:
Date/time of amount:
Employee
Name:
Orders management Employee
Inventory Employee

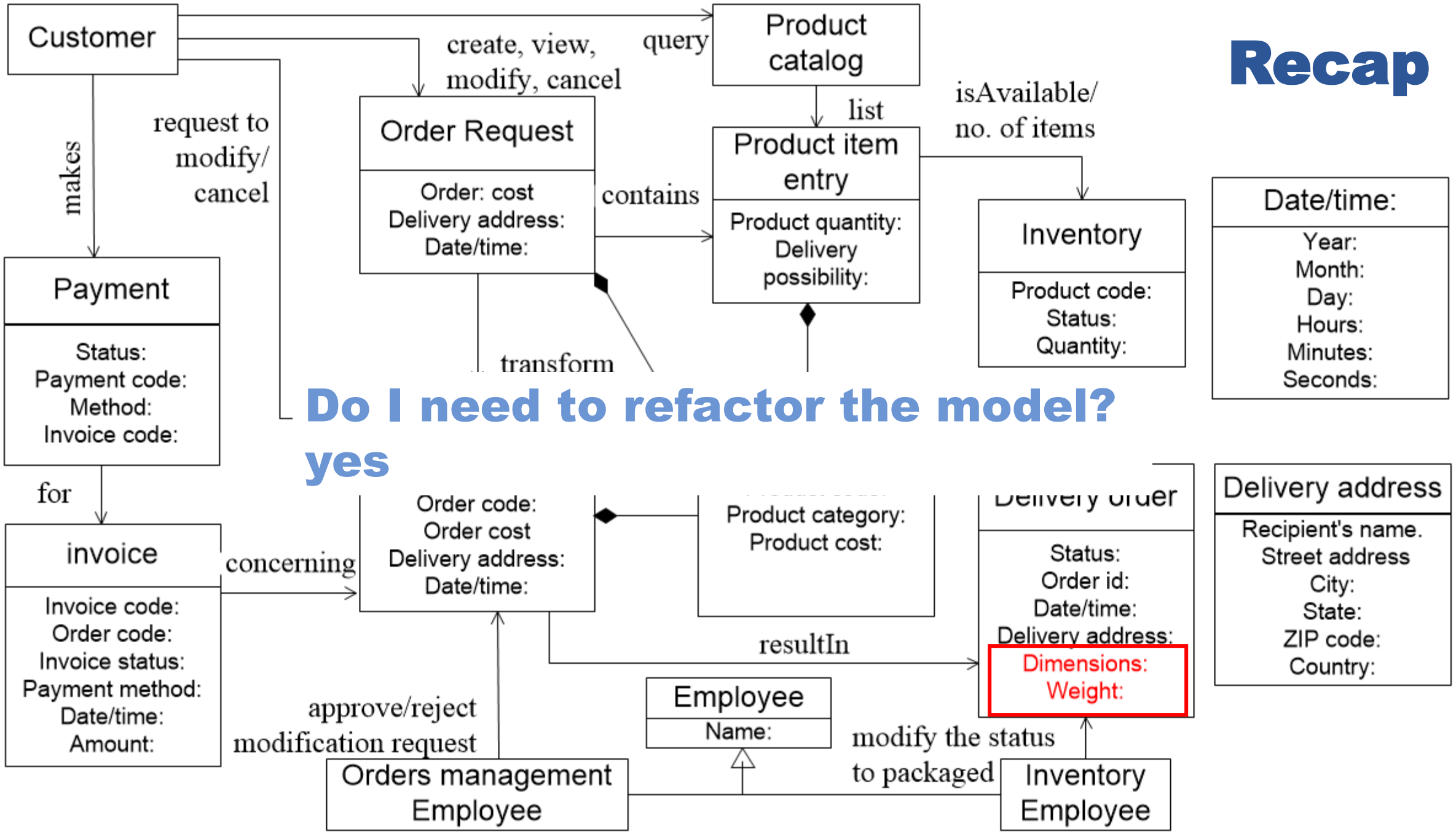
Recap


Customer
Product catalog
Order Request
Product code:
Product category:
Product cost:
Order: cost
Delivery address:
Date/time:
Product item entry
Product code:
Product category:
Product cost:
Product quantity:
Delivery possibility:
Inventory
Product code:
Status:
Quantity:
Payment
Status:
Payment code:
Method:
Invoice code:
Purchase Order
Order status:
Order code:
Product code:
Product category:
Product cost:
Order cost
Delivery address:
Date/time:
Product item
Product code:
Product category:
Product cost:
Delivery order
Status:
Order id:
Date/time:
Delivery address:
Delivery address
Recipient's name.
Street address
City:
State:
ZIP code:
Country:
invoice
Invoice code:
Order code:
Invoice status:
Payment method:
Date/time of amount:
Employee
Name:
Orders management Employee
Inventory Employee

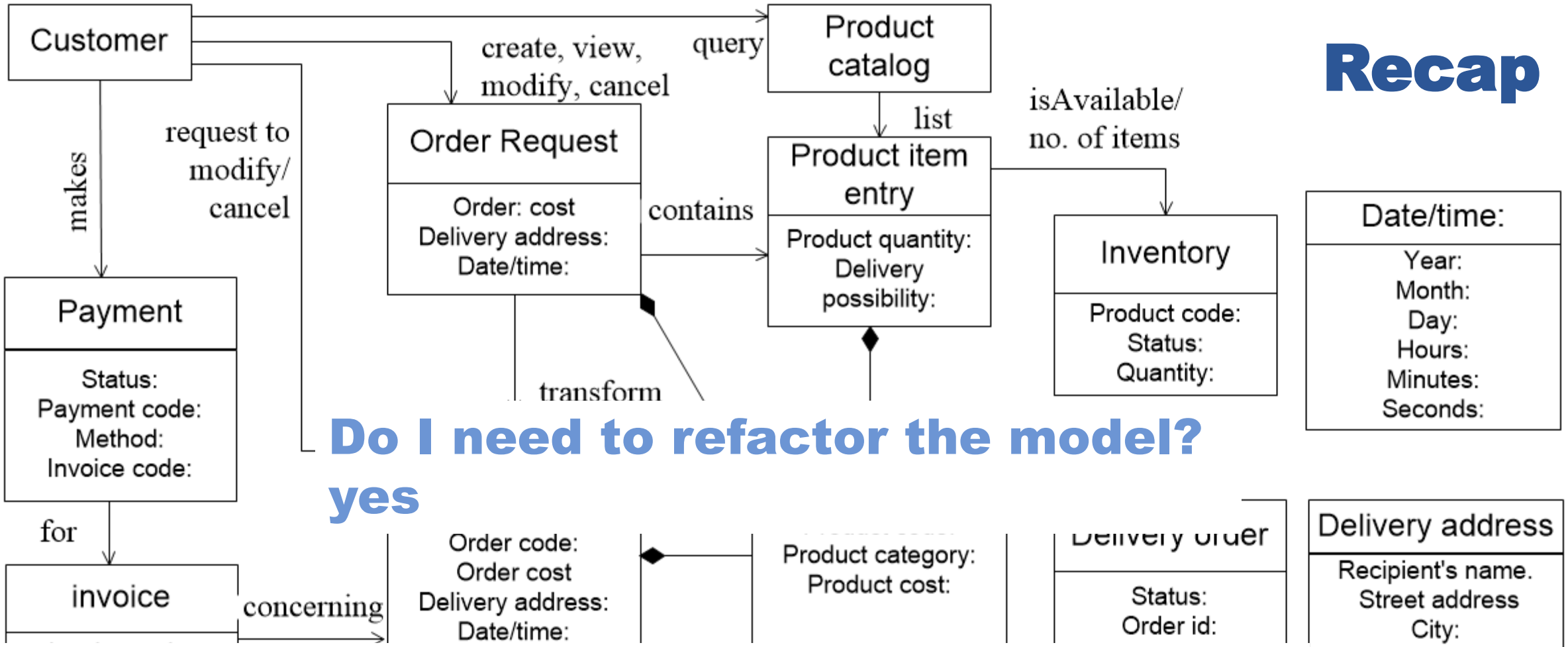
Recap


Customer
Product catalog
query
create, view, modify, cancel
makes
request to modify/ cancel
Order Request
Order: cost
Delivery address:
Date/time:
contains
Product item entry
Product quantity:
Delivery possibility:
list
isAvailable/ no. of items
Inventory
Product code:
Status:
Quantity:
Date/time:
Year:
Month:
Day:
Hours:
Minutes:
Seconds:
Payment
Status:
Payment code:
Method:
Invoice code:
transform
Purchase Order
Order status:
Order code:
Order cost
Delivery address:
Date/time:
Product item
Product code:
Product category:
Product cost:
for
invoice
Invoice code:
Order code:
Invoice status:
Payment method:
Date/time:
Amount:
concerning
Delivery order
Status:
Order id:
Date/time:
Delivery address:
Delivery address
Recipient's name.
Street address
City:
State:
ZIP code:
Country:
resultIn
Employee
Name:
approve/reject modification request
modify the status to packaged
Orders management Employee
Inventory Employee

Recap


Customer
create, view, modify, cancel
query
Product catalog
request to modify/ cancel
makes
Order Request
Order: cost
Delivery address:
Date/time:
contains
list
Product item entry
Product quantity:
Delivery possibility:
isAvailable/ no. of items
Inventory
Product code:
Status:
Quantity:
Date/time:
Year:
Month:
Day:
Hours:
Minutes:
Seconds:
Payment
Status:
Payment code:
Method:
Invoice code:
transform
Do I need to refactor the model?
yes
for
invoice
Invoice code:
Order code:
Invoice status:
Payment method:
Date/time:
Amount:
concerning
Order code:
Order cost
Delivery address:
Date/time:
Product category:
Product cost:
Delivery order
Status:
Order id:
Date/time:
Delivery address:
Dimensions:
Weight:
Delivery address
Recipient's name.
Street address
City:
State:
ZIP code:
Country:
resultIn
approve/reject modification request
Employee
Name:
modify the status to packaged
Orders management Employee
Inventory Employee

Recap

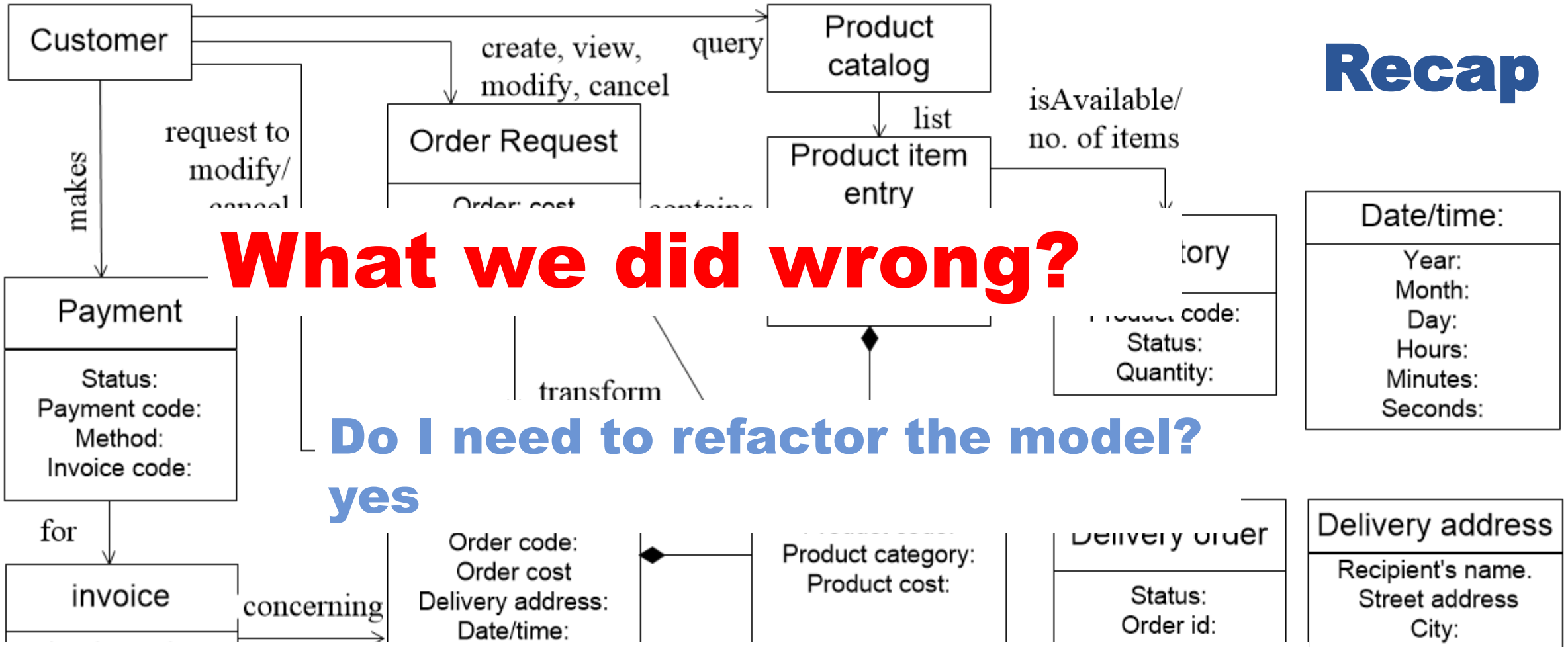


Do I need to refactor the model?
yes

The most important thing is that after all this analysis, it seems that we did not have a full understanding of

what a product is!!

Recap



What we did wrong?

Do I need to refactor the model?

yes

The most important thing is that after all this analysis, it seems that we did not have a full understanding of

what a product is!!

Domain-driven design (DDD)

DDD is a software ***design approach*** that focus on modelling software to match ***a domain*** according to input from domain's experts[1].

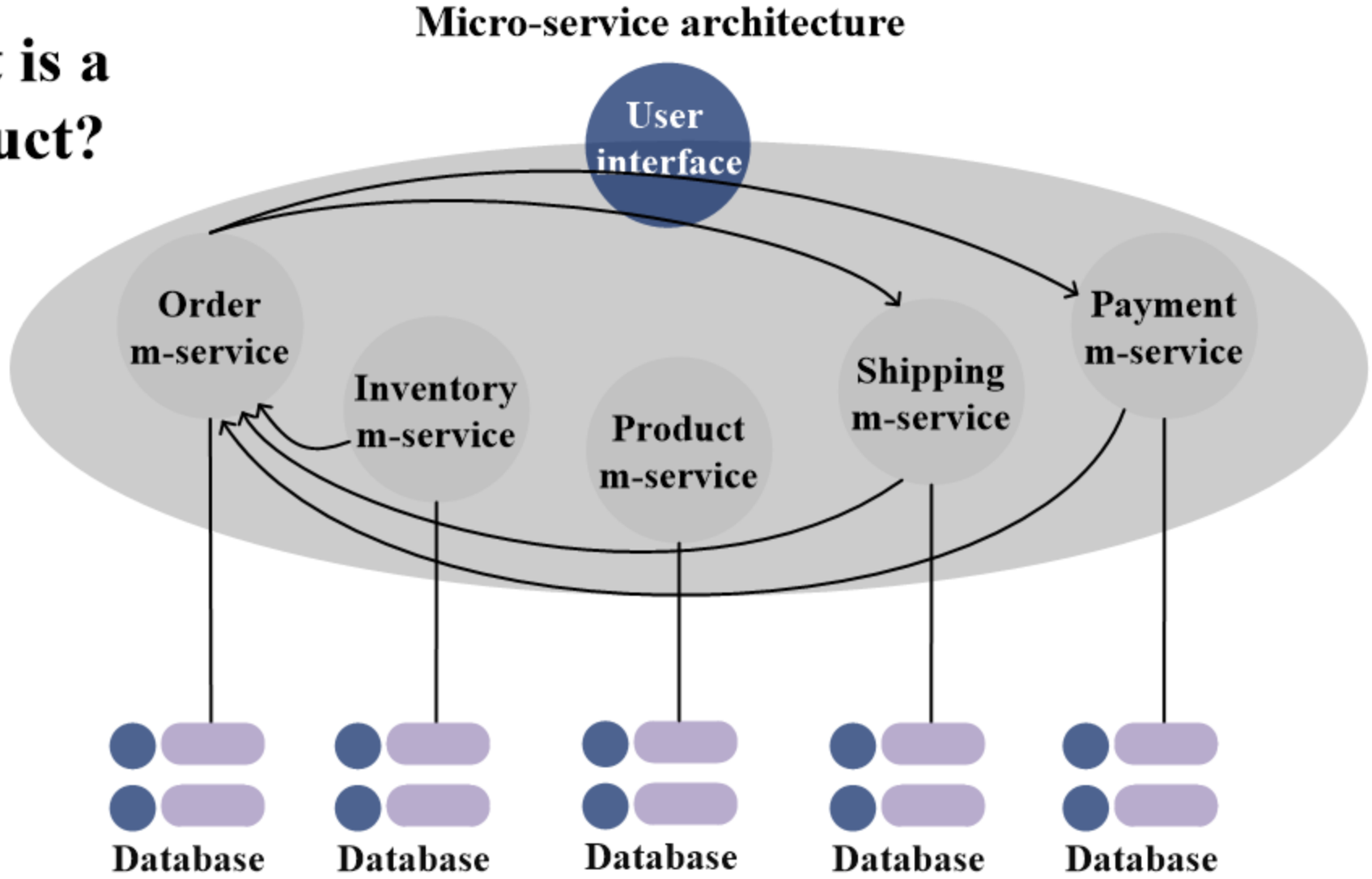
DDD is used for developing software for ***complex software systems***.

Applying the ***DDD*** require deep understanding of the ***domain of discloser***.

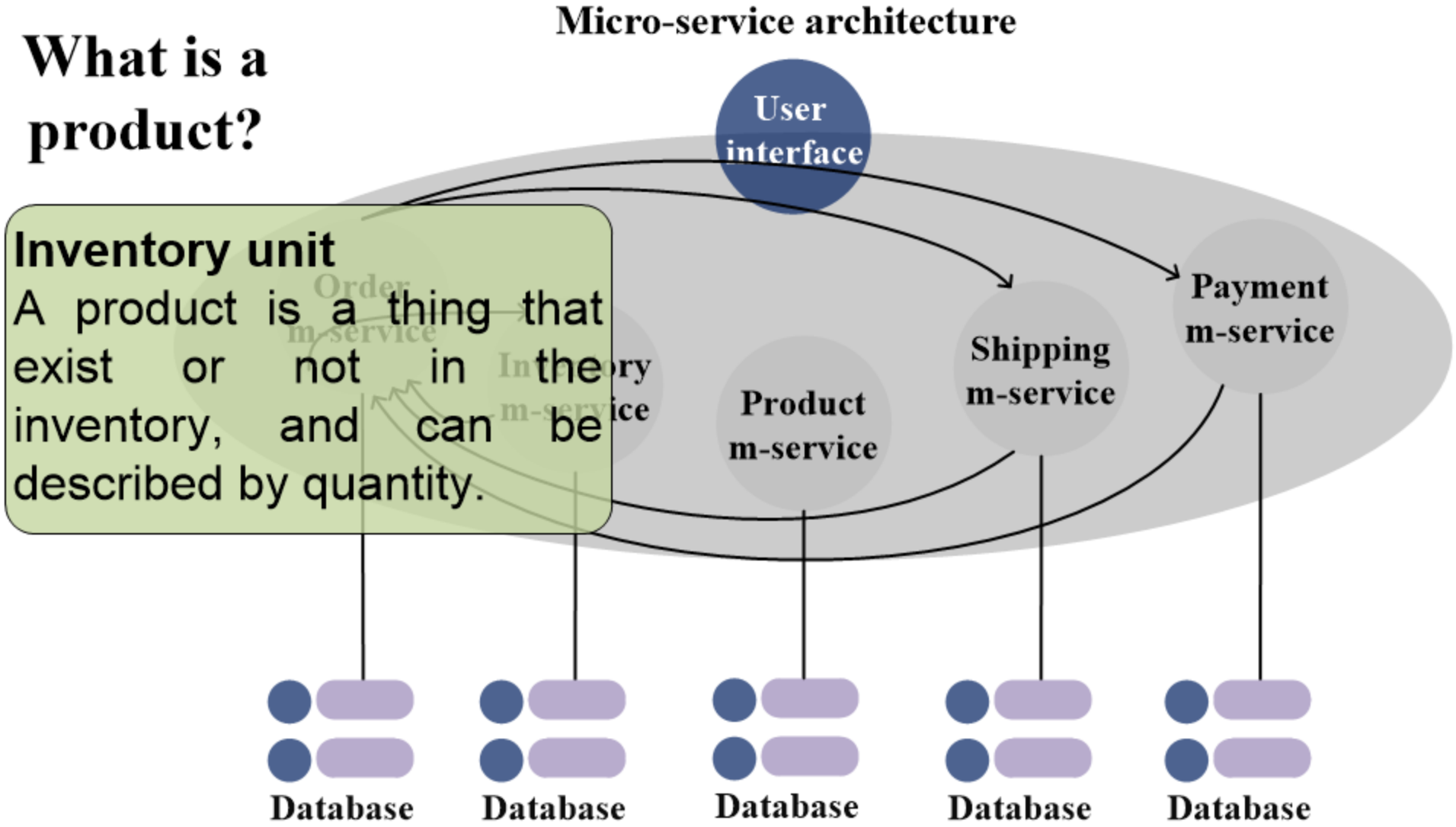
Vernon, Vaughn (2013). Implementing Domain-Driven Design. Upper Sadle River, NJ: Addison-Wesley. p. 3. ISBN 978-0-321-83457-7.

Why do we need the DDD?

What is a product?

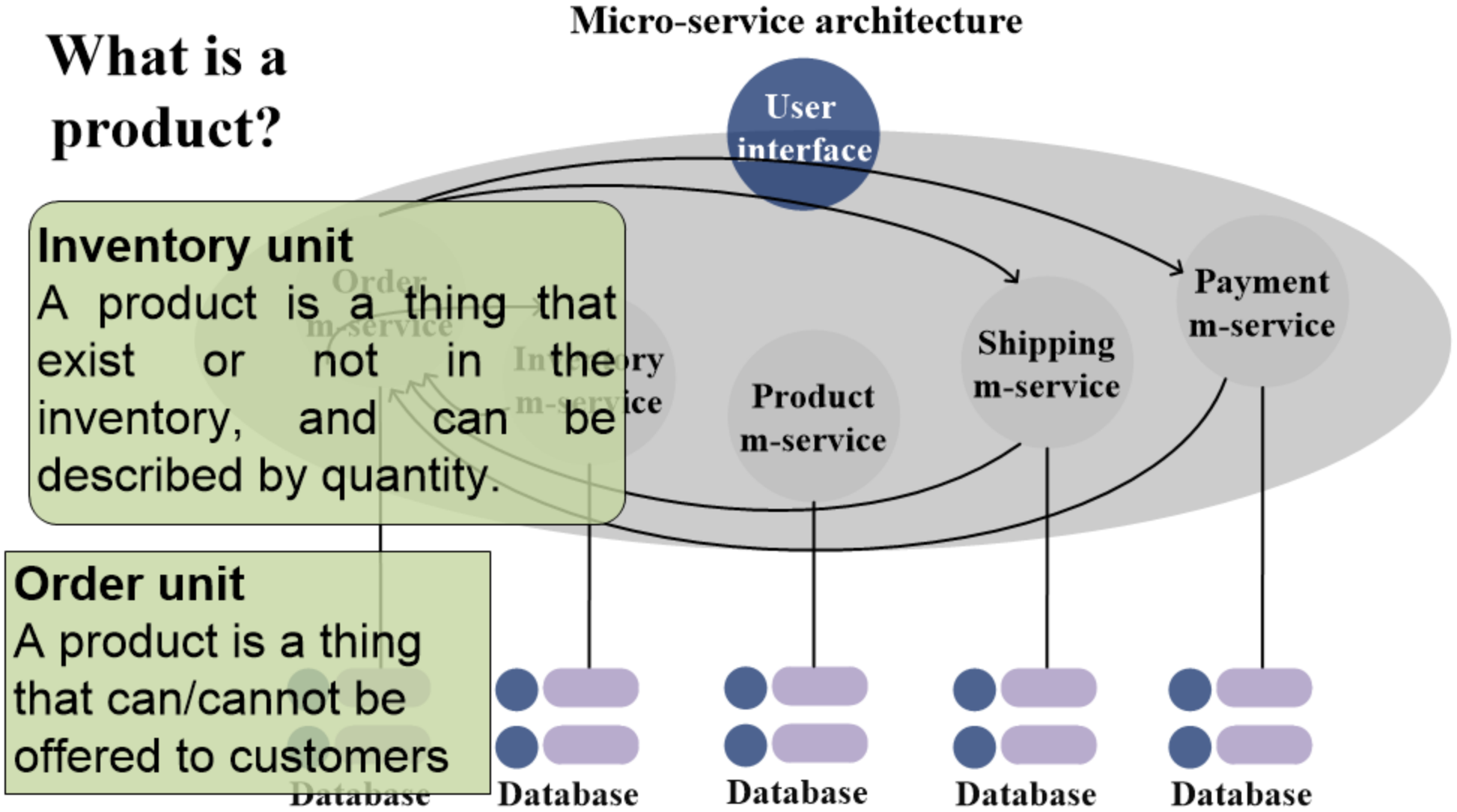


Why do we need the DDD?



Why do we need the DDD?

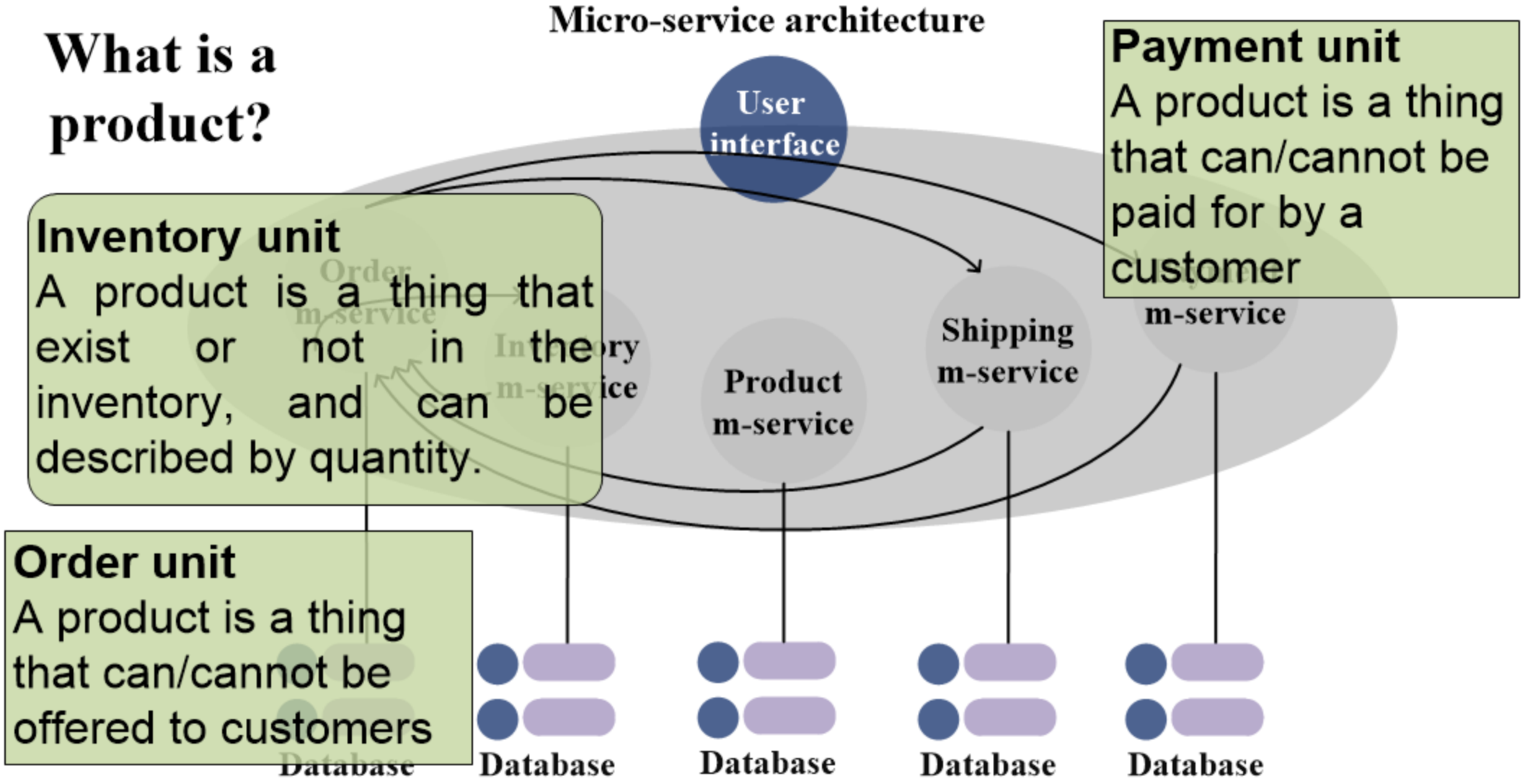
What is a product?



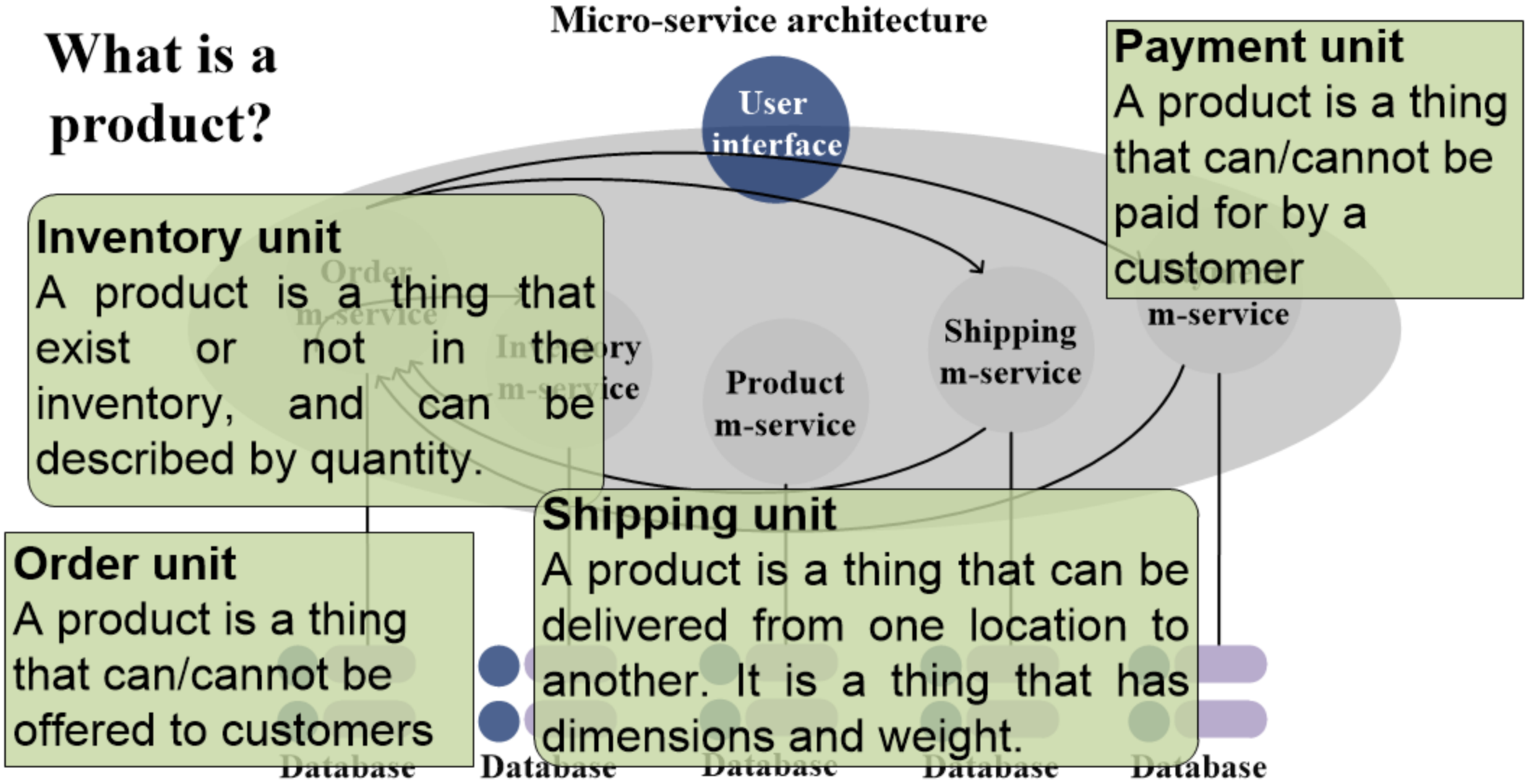
Inventory unit
A product is a thing that exist or not in the inventory, and can be described by quantity.

Order unit
A product is a thing that can/cannot be offered to customers

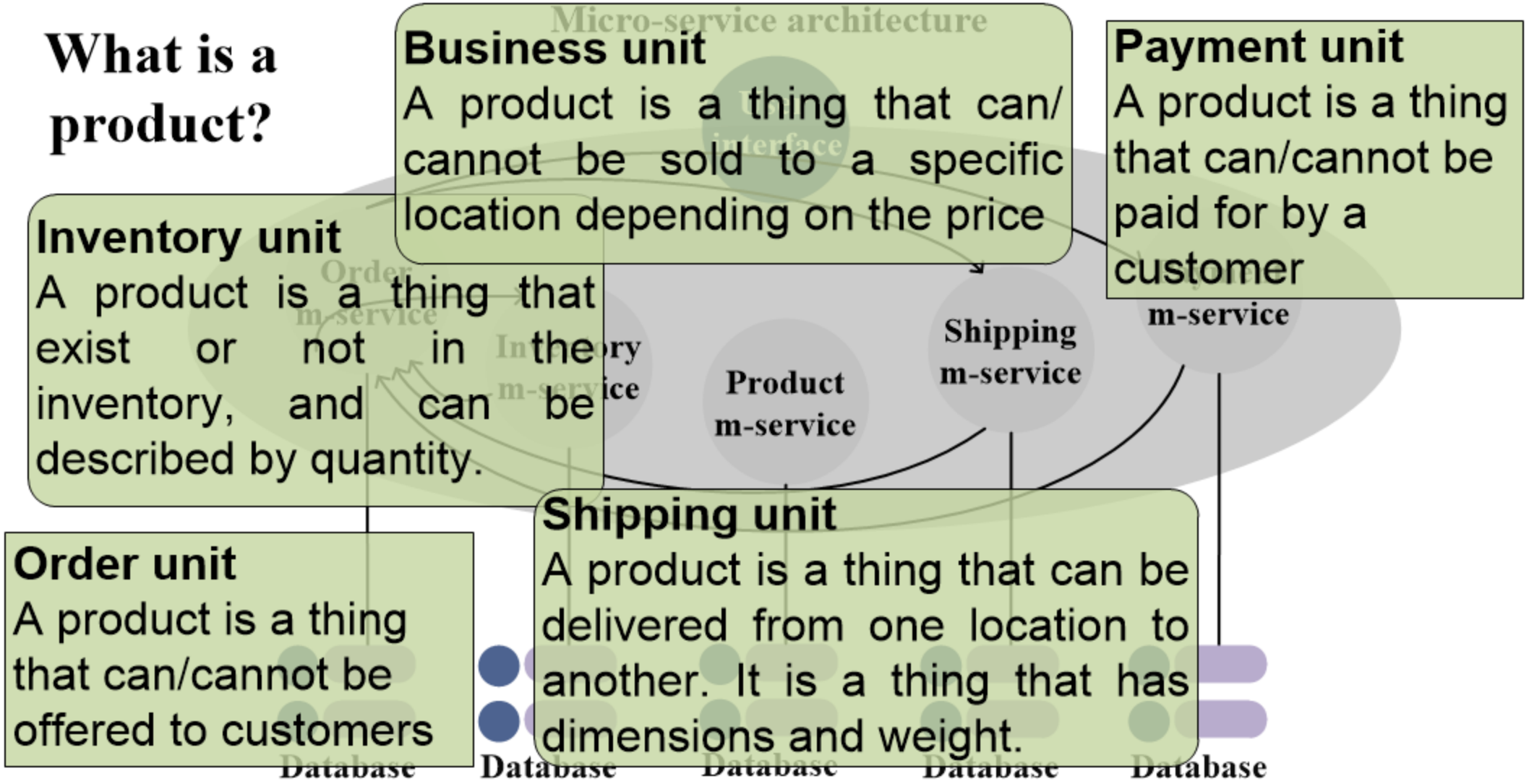
Why do we need the DDD?



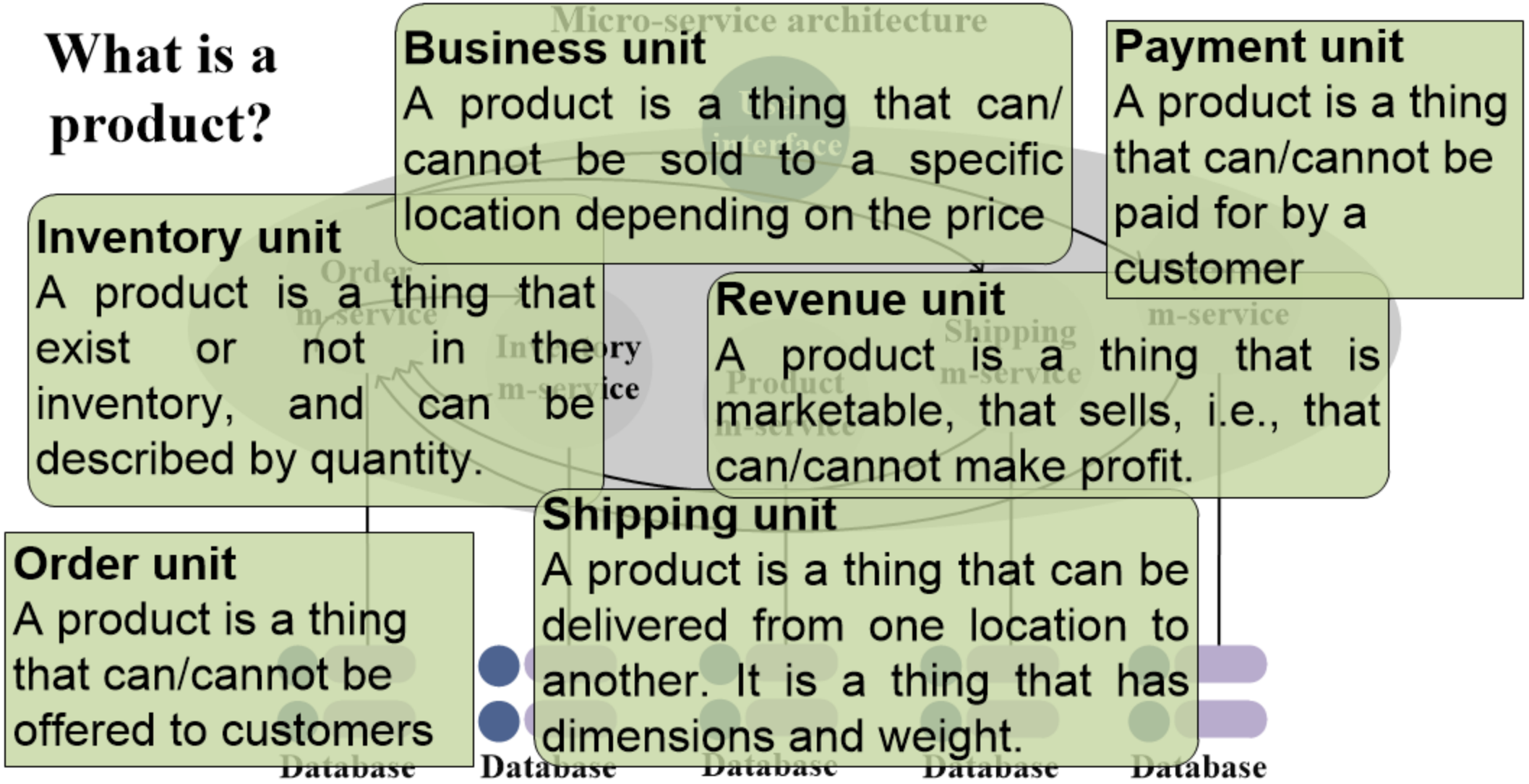
Why do we need the DDD?



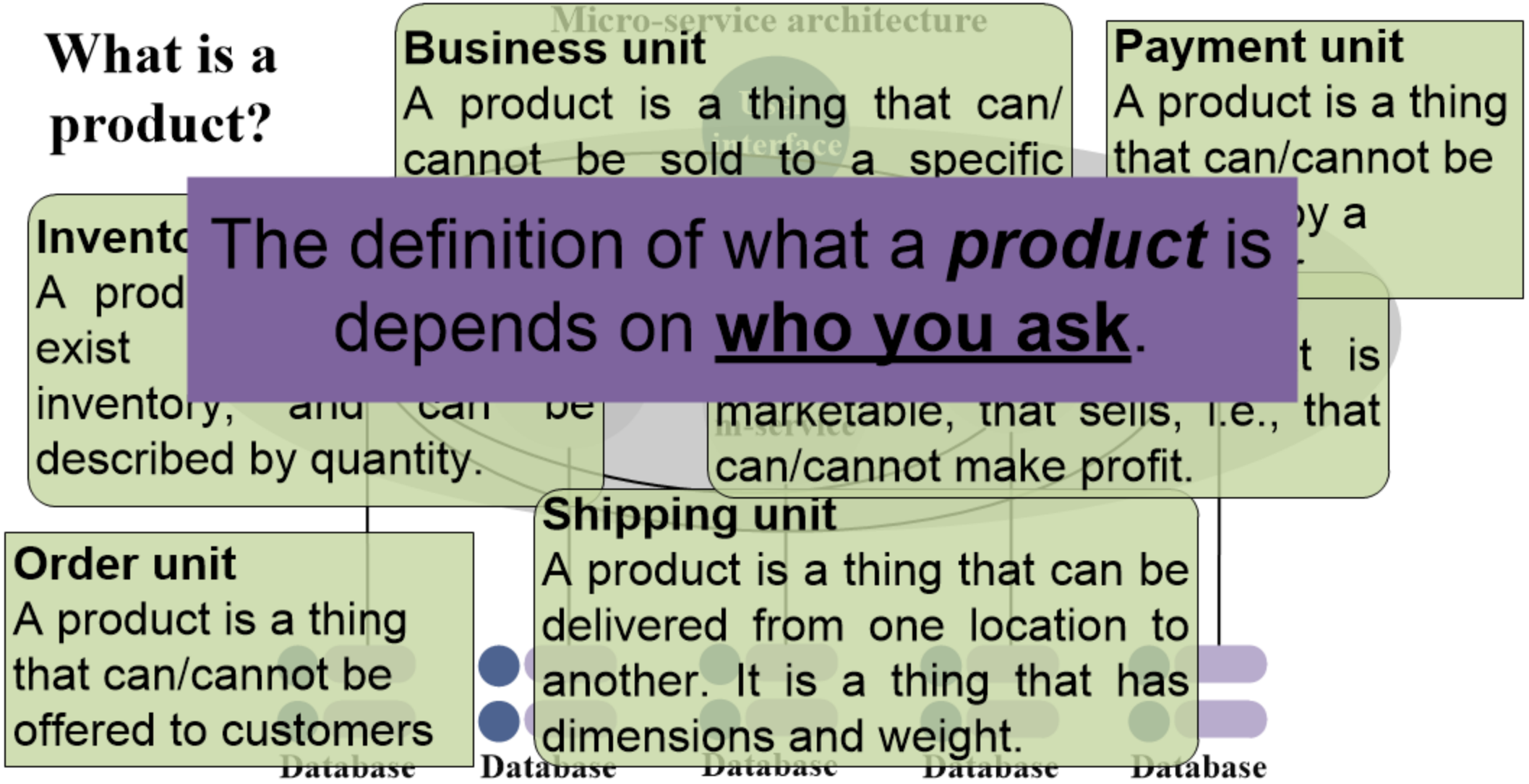
Why do we need the DDD?



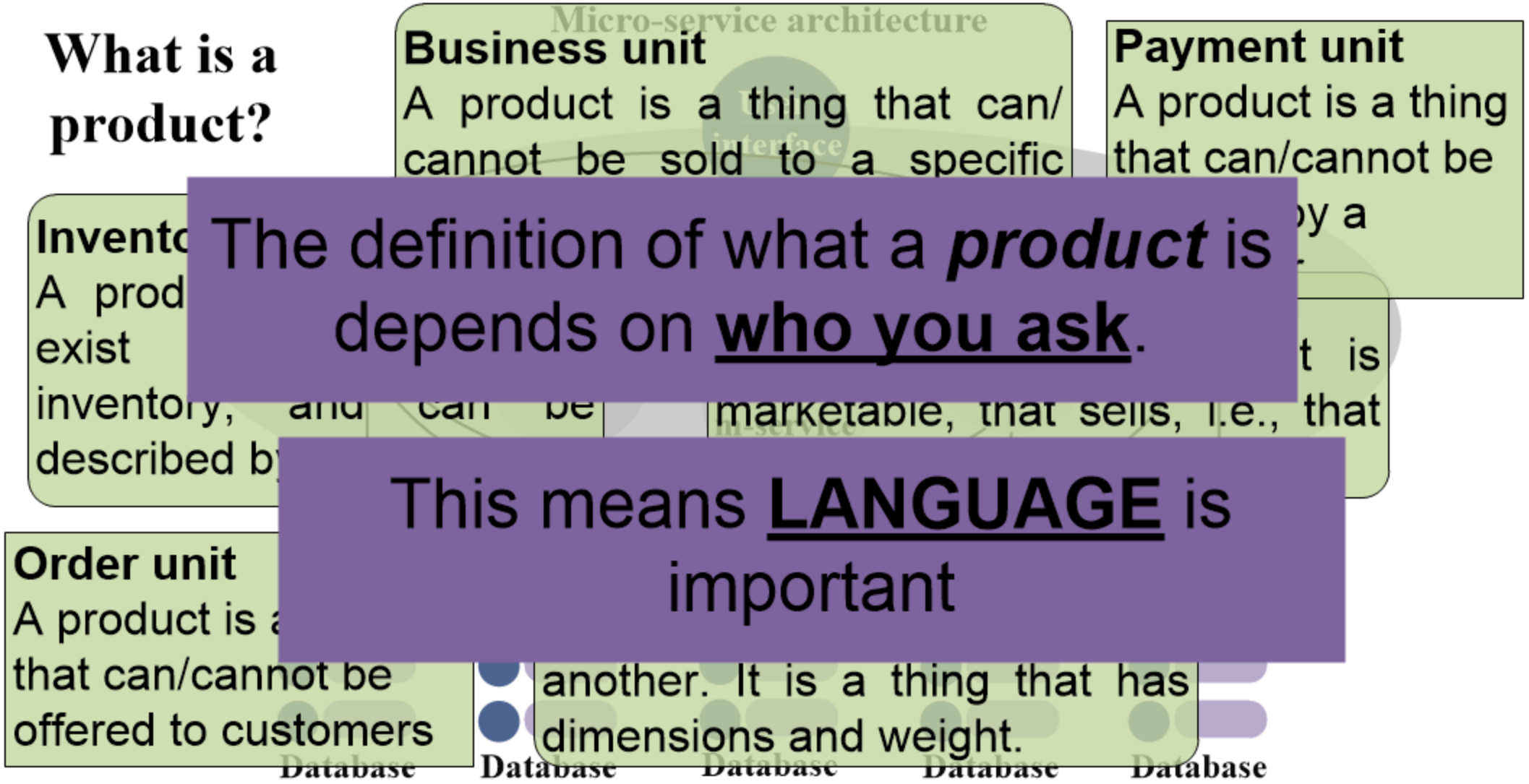
Why do we need the DDD?



Why do we need the DDD?



Why do we need the DDD?



The definition of what a ***product*** is depends on **who you ask**.

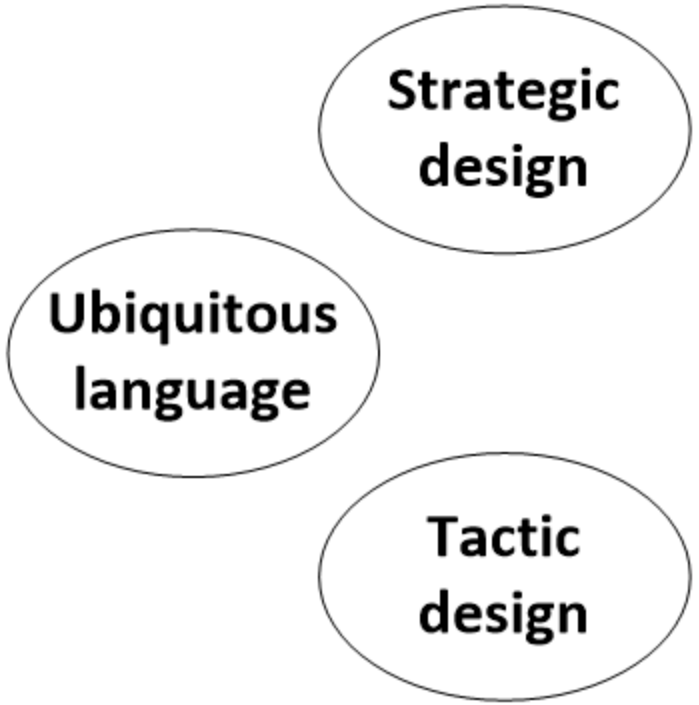
This means **LANGUAGE** is important

Domain-driven design (DDD)

Strategic design defines the bounded contexts and the Context Maps, and contributes the Ubiquitous Language.

Tactic design provides a tool to link Strategic design to implementation.

Ubiquitous language a common language that connects all the parts of the design.

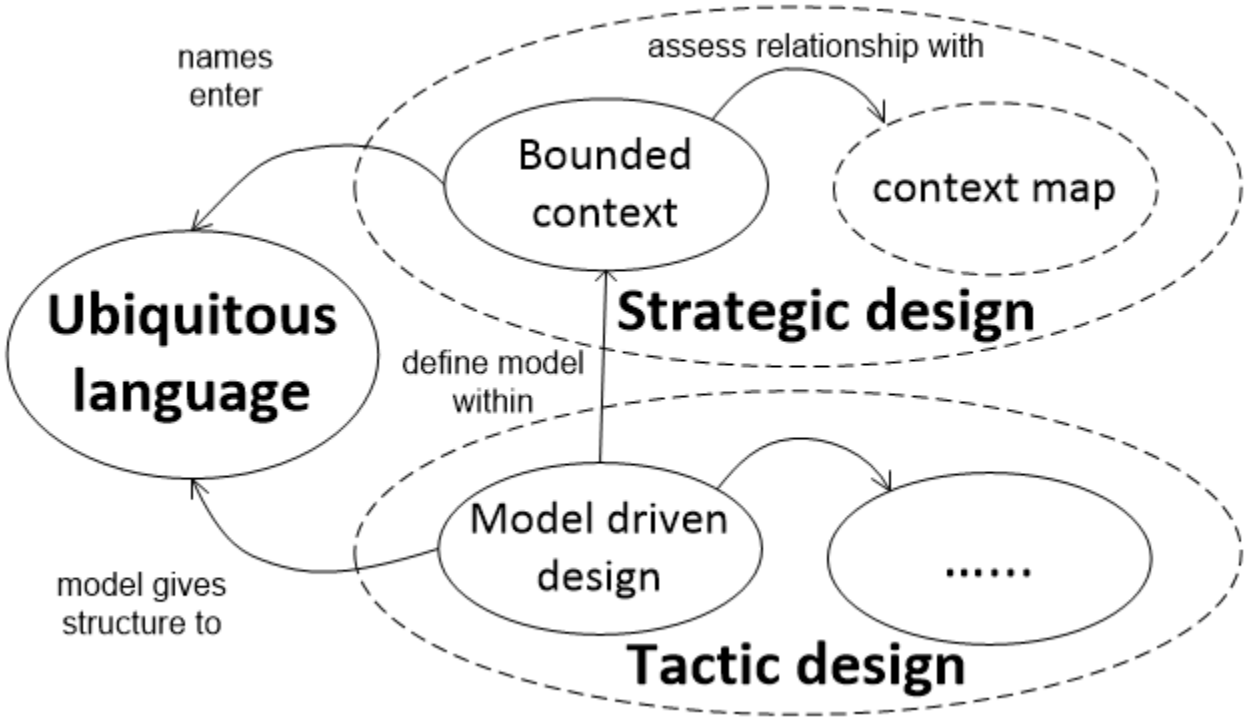


Domain-driven design (DDD)

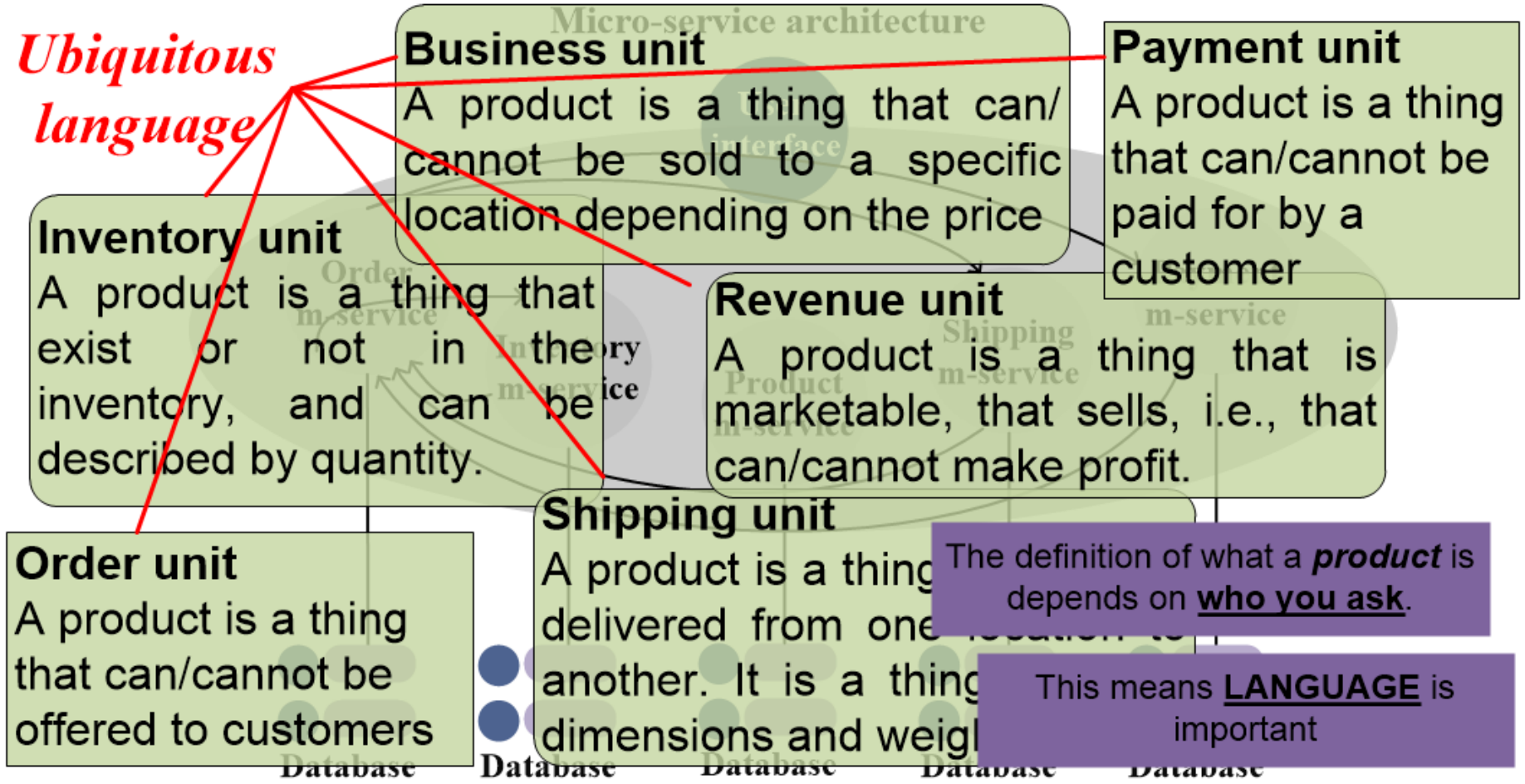
Strategic design defines the bounded contexts and the Context Maps, and contributes the Ubiquitous Language.

Tactic design provides a tool to link Strategic design to implementation.

Ubiquitous language a common language that connects all the parts of the design.

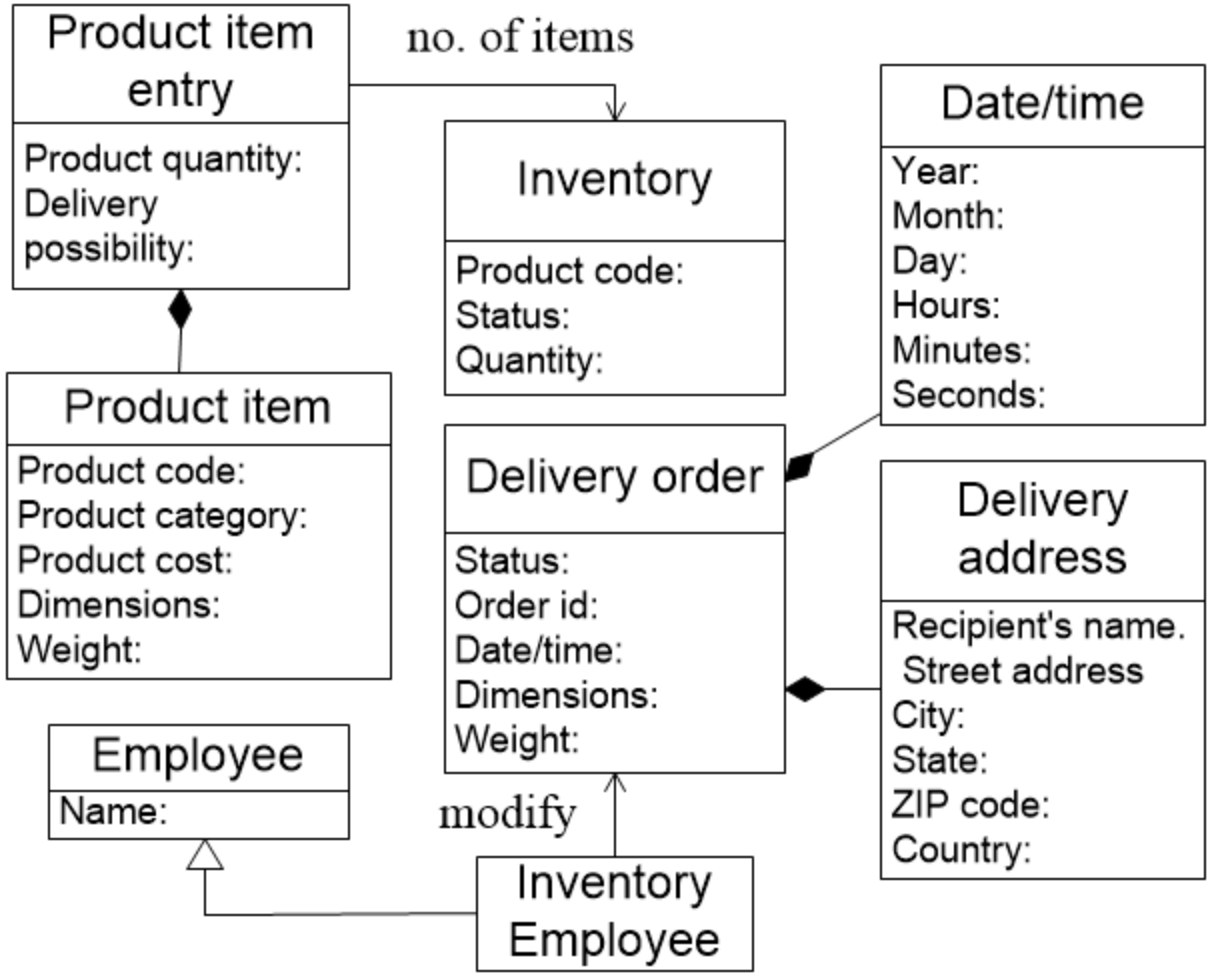


DDD - Ubiquitous language



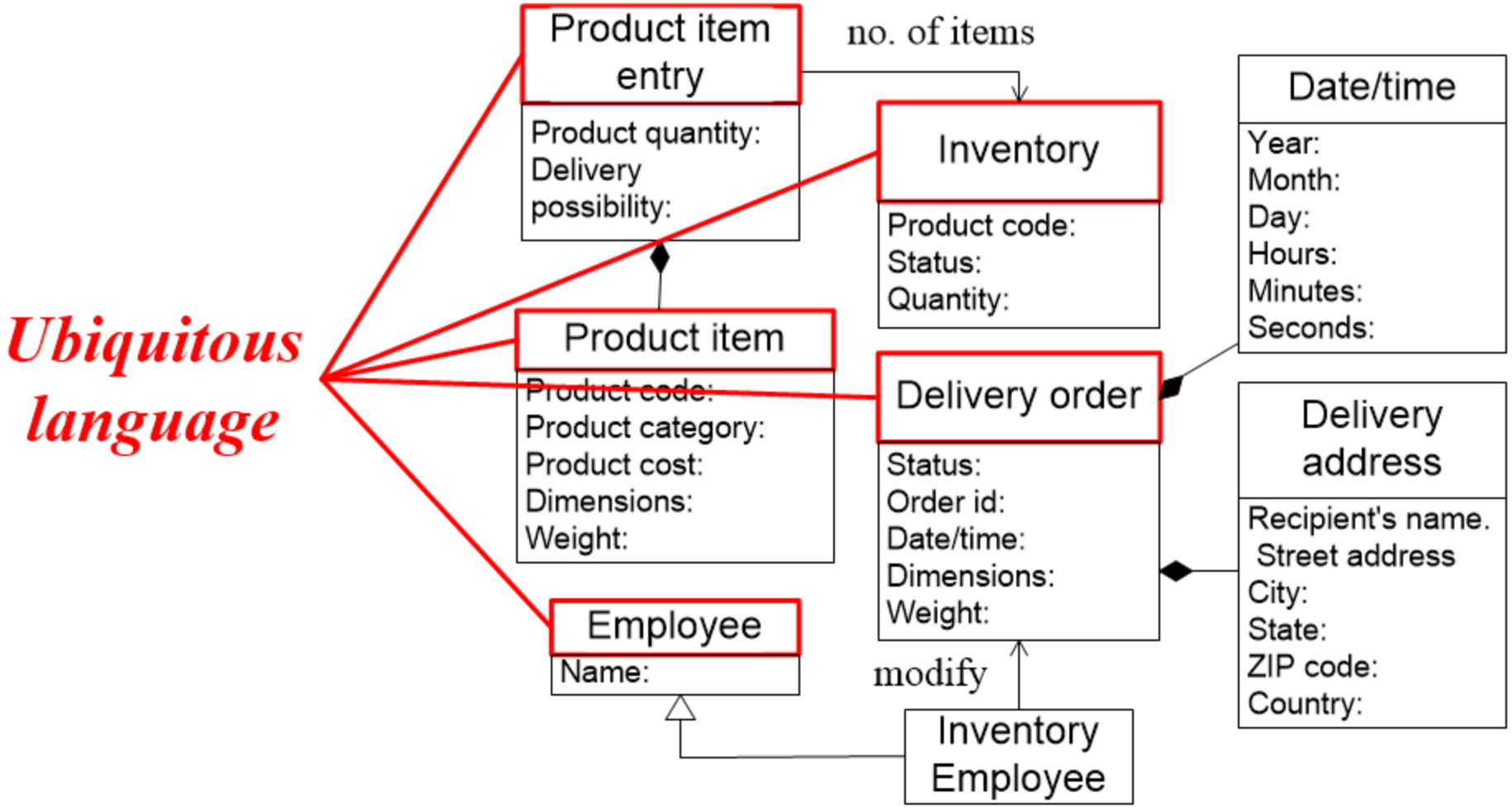
Ubiquitous language a common language that connects all the parts of the design.

DDD - Ubiquitous language



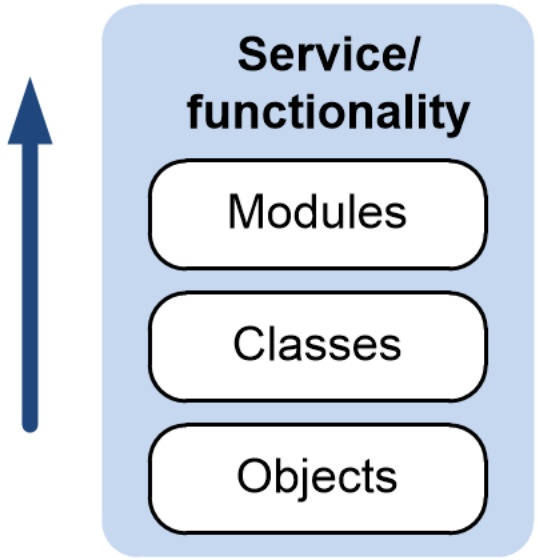
Ubiquitous language a common language that connects all the parts of the design.

DDD - Ubiquitous language



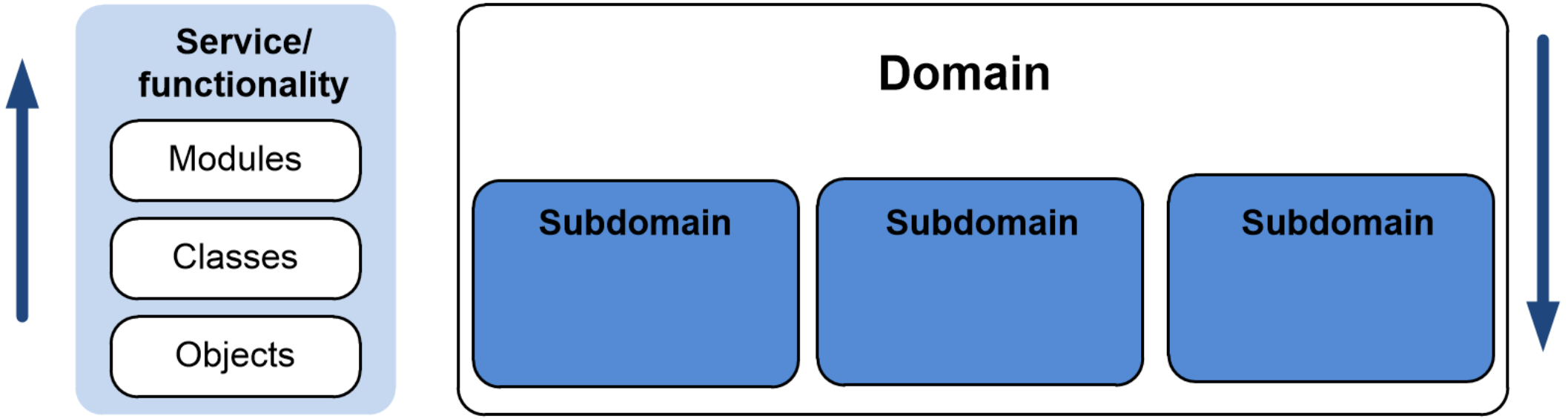
Ubiquitous language a common language that connects all the parts of the design.

Domain-driven design (DDD)



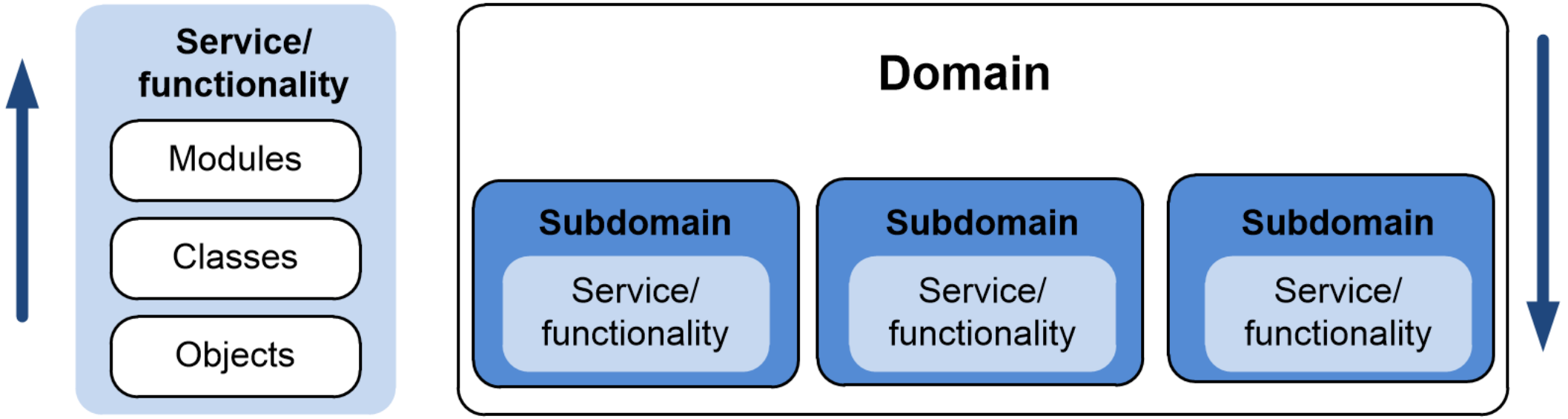
The **bottom-up approach** works by starting with **individual components** and building **up** to the **larger system**.

Domain-driven design (DDD)



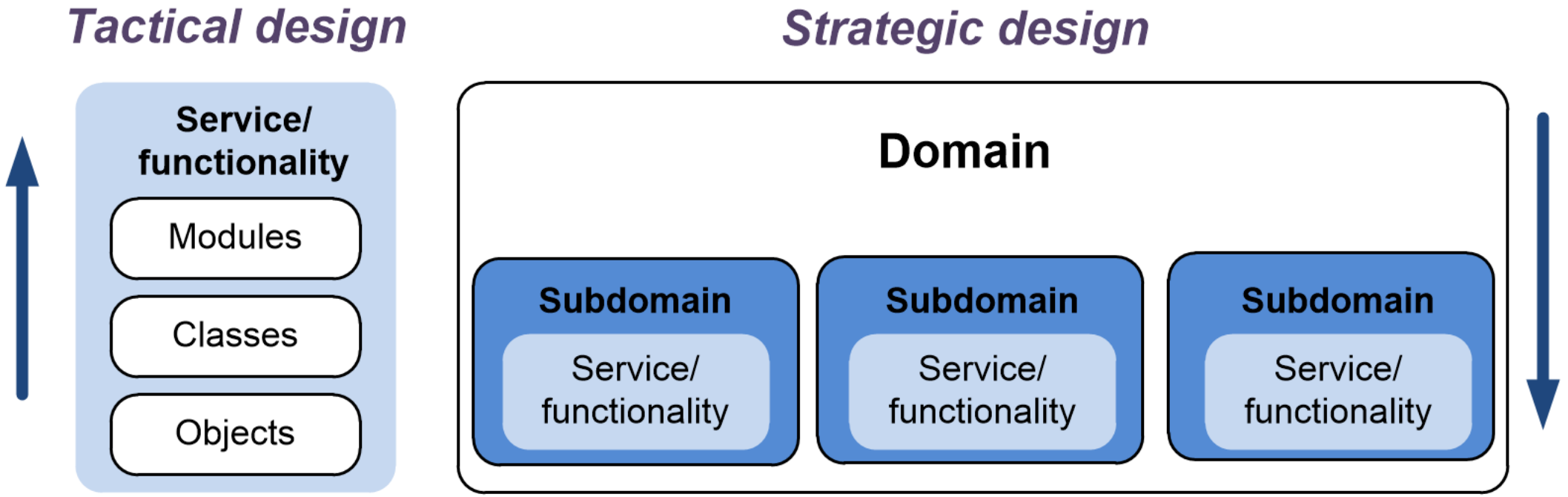
DDD is an approach that “design” the system from ***top to down.***

Domain-driven design (DDD)



DDD is an approach that “design” the system from ***top to down.***

Domain-driven design (DDD)



DDD is an approach that "design" the system from ***top to down.***

DDD - Strategic design

Bounded context is the boundary within a domain where a particular **domain model applies**.

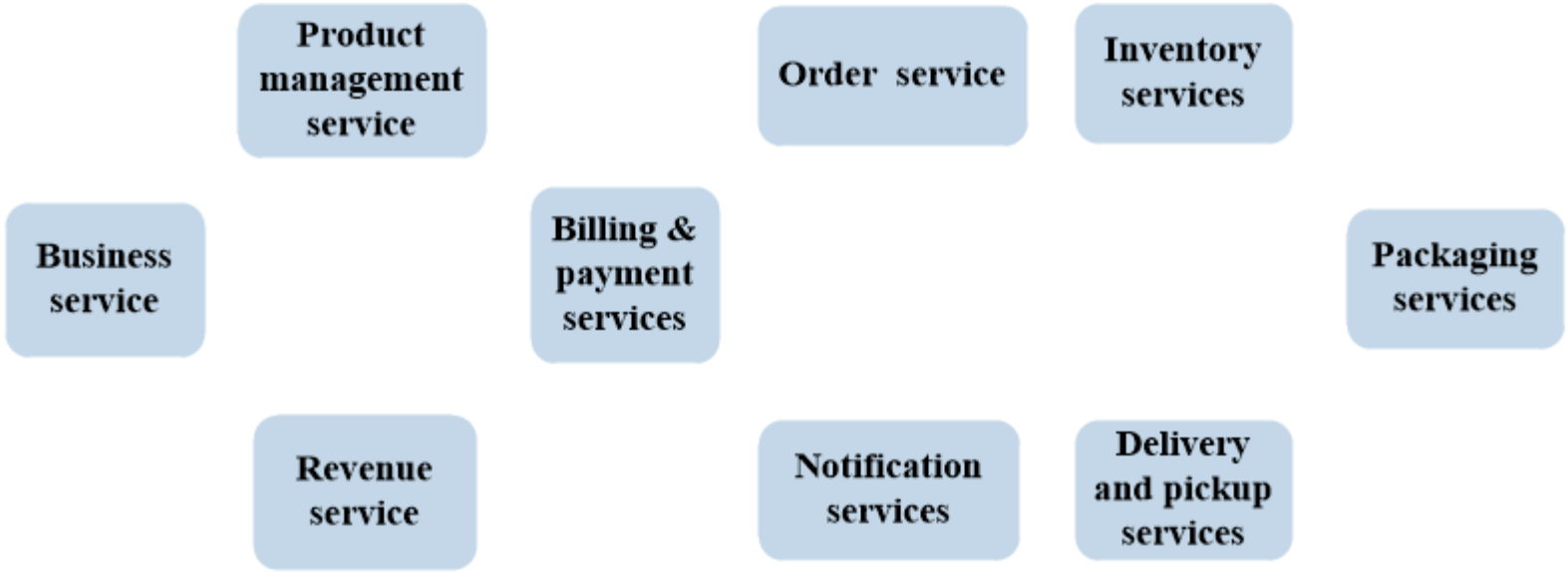
A key aspect to define a **bounded context** is that, it should be able to **function by itself**.

Note: bounded contexts are not necessarily isolated from one another.

DDD - Strategic design

Bounded context is the boundary within a domain where a particular **domain model applies**.

A key aspect to define a **bounded context** is that, it should be able to **function by itself**.

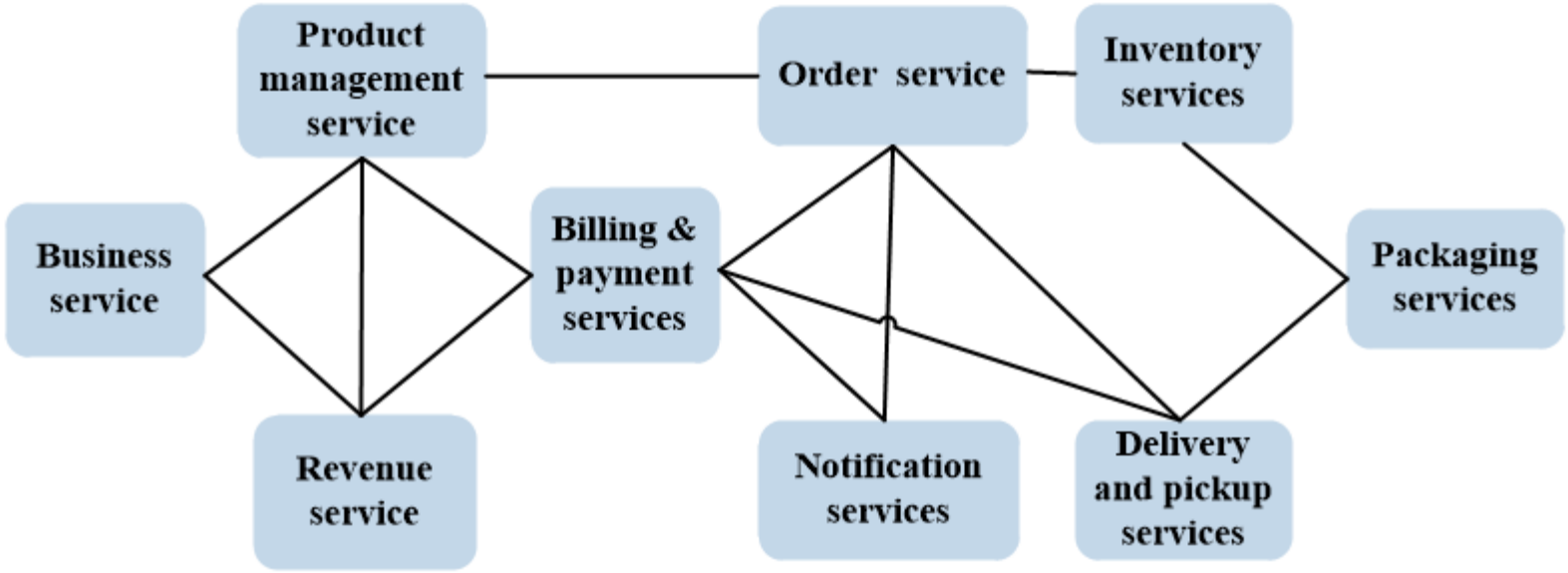


Note: bounded contexts are not necessarily isolated from one another.

DDD - Strategic design

Bounded context is the boundary within a domain where a particular **domain model applies**.

A key aspect to define a **bounded context** is that, it should be able to **function by itself**.

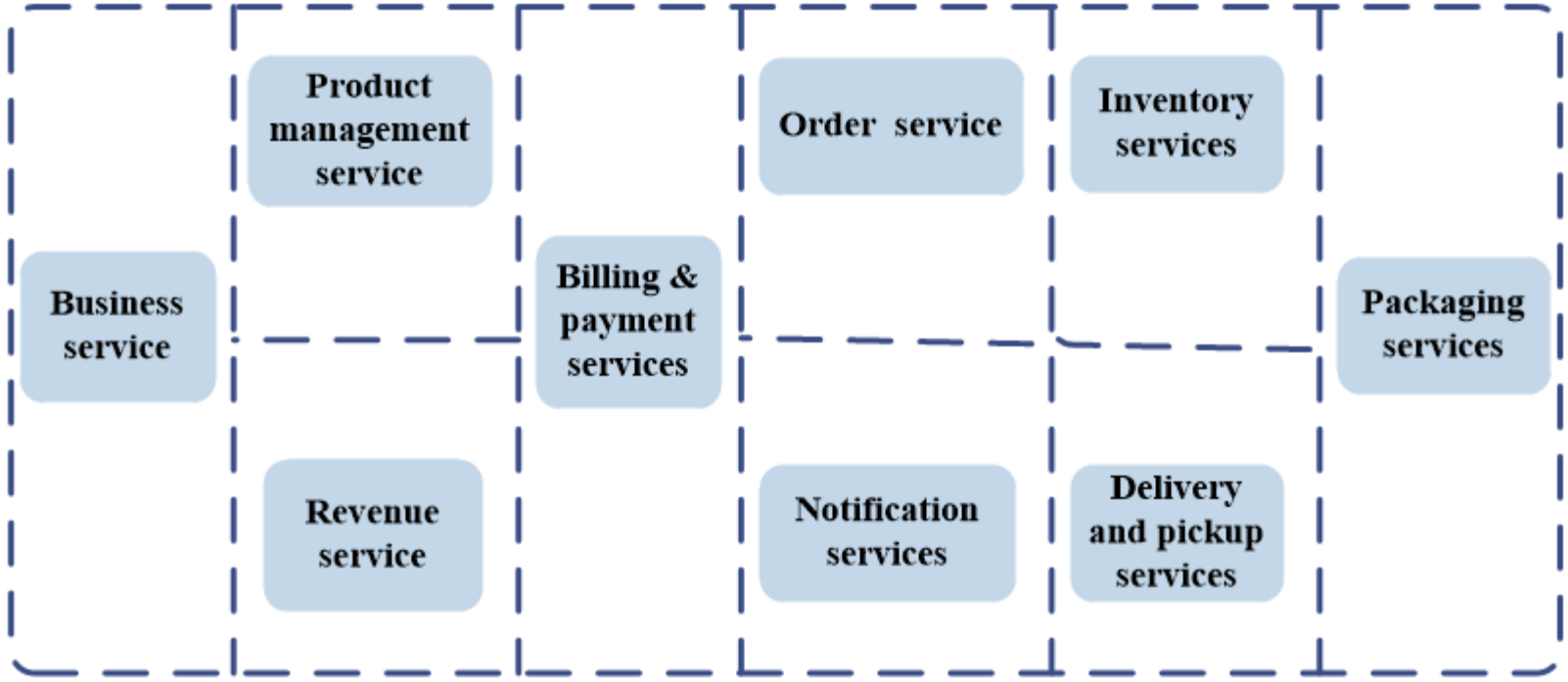


Note: bounded contexts are not necessarily isolated from one another.

DDD - Strategic design

Bounded context is the boundary within a domain where a particular **domain model applies**.

A key aspect to define a **bounded context** is that, it should be able to **function by itself**.

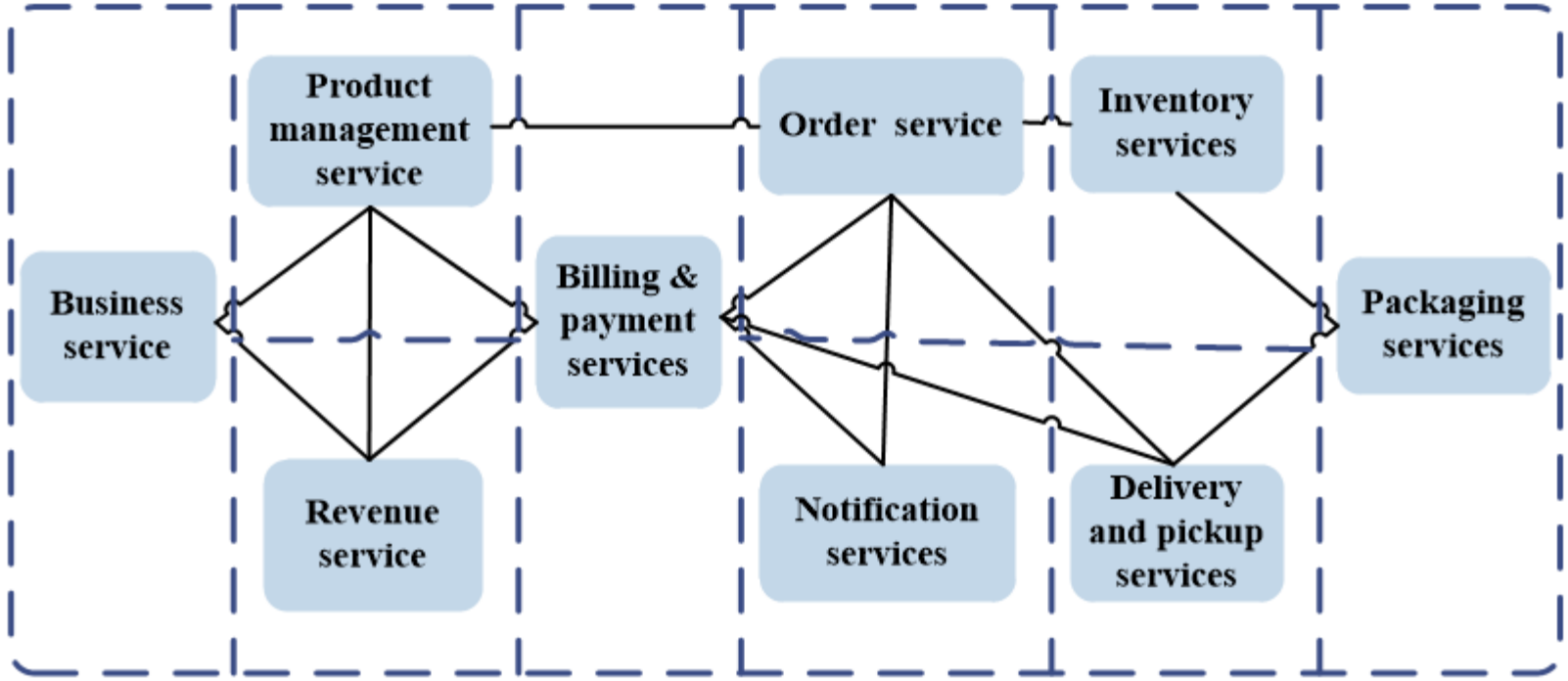


Note: bounded contexts are not necessarily isolated from one another.

DDD - Strategic design

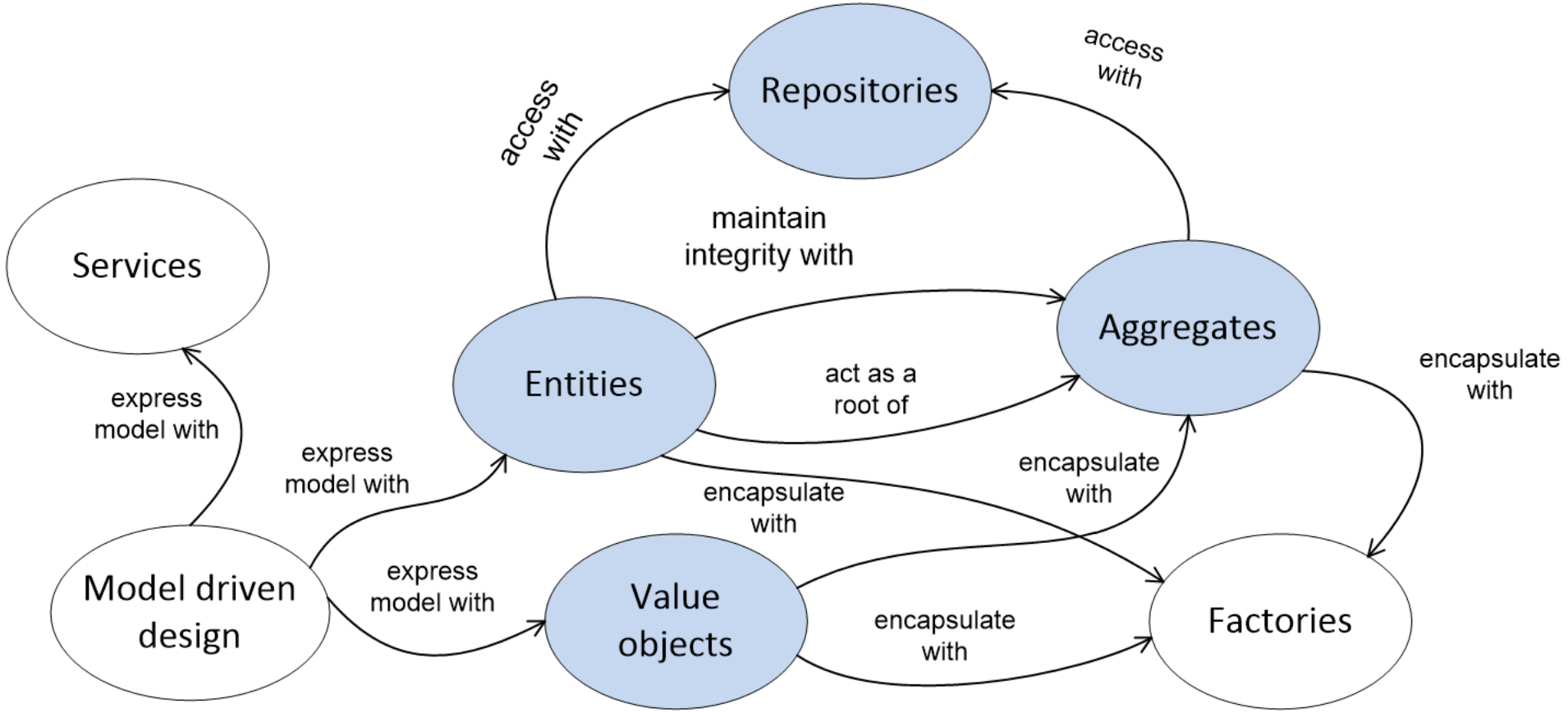
Bounded context is the boundary within a domain where a particular **domain model applies**.

A key aspect to define a **bounded context** is that, it should be able to **function by itself**.



Context Maps captures the relationships between bounded contexts.

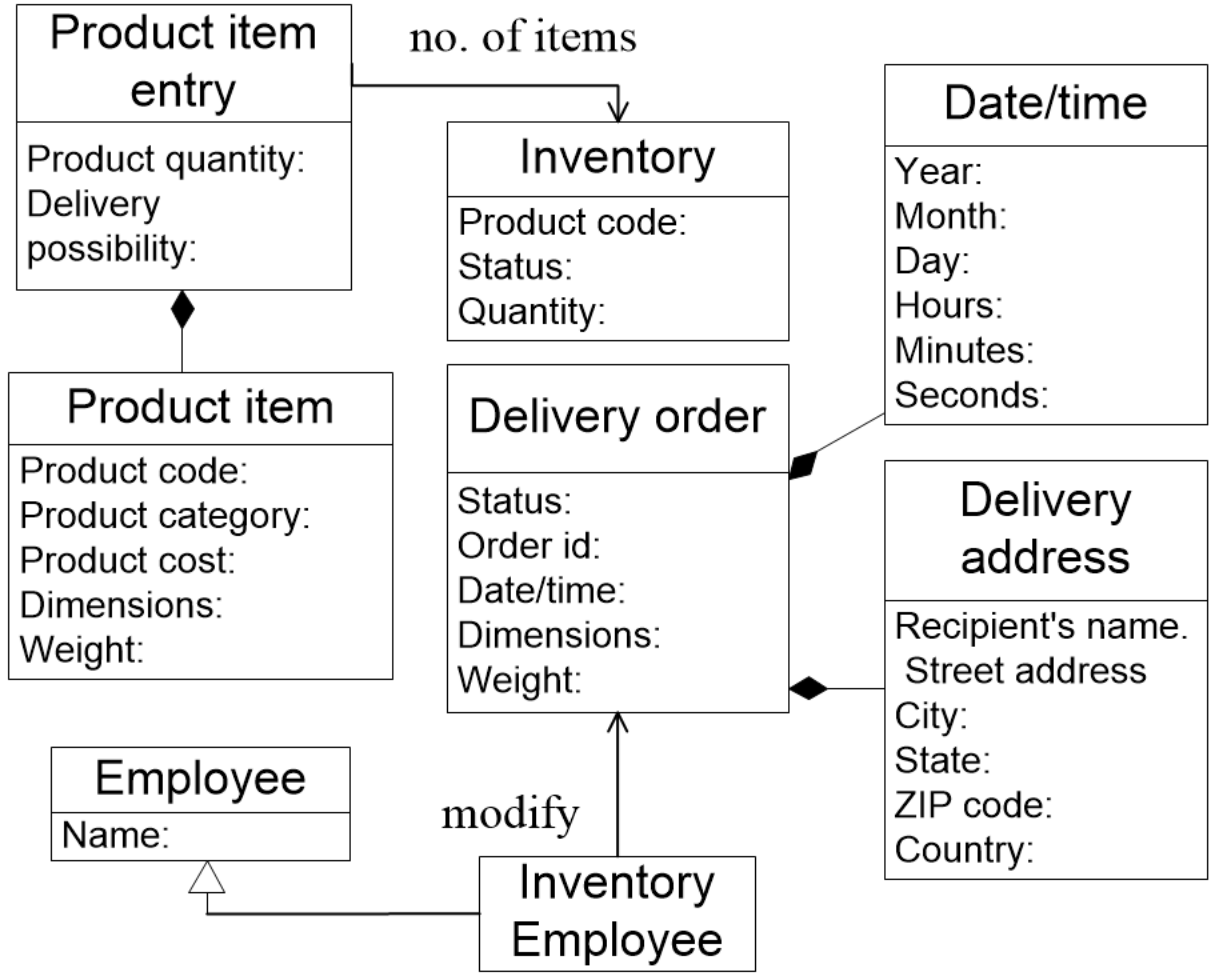
DDD - Tactic design



DDD - Tactic design

An entity is an object that is identified by its consistent thread of continuity, as opposed to traditional objects, which are defined by their attributes.

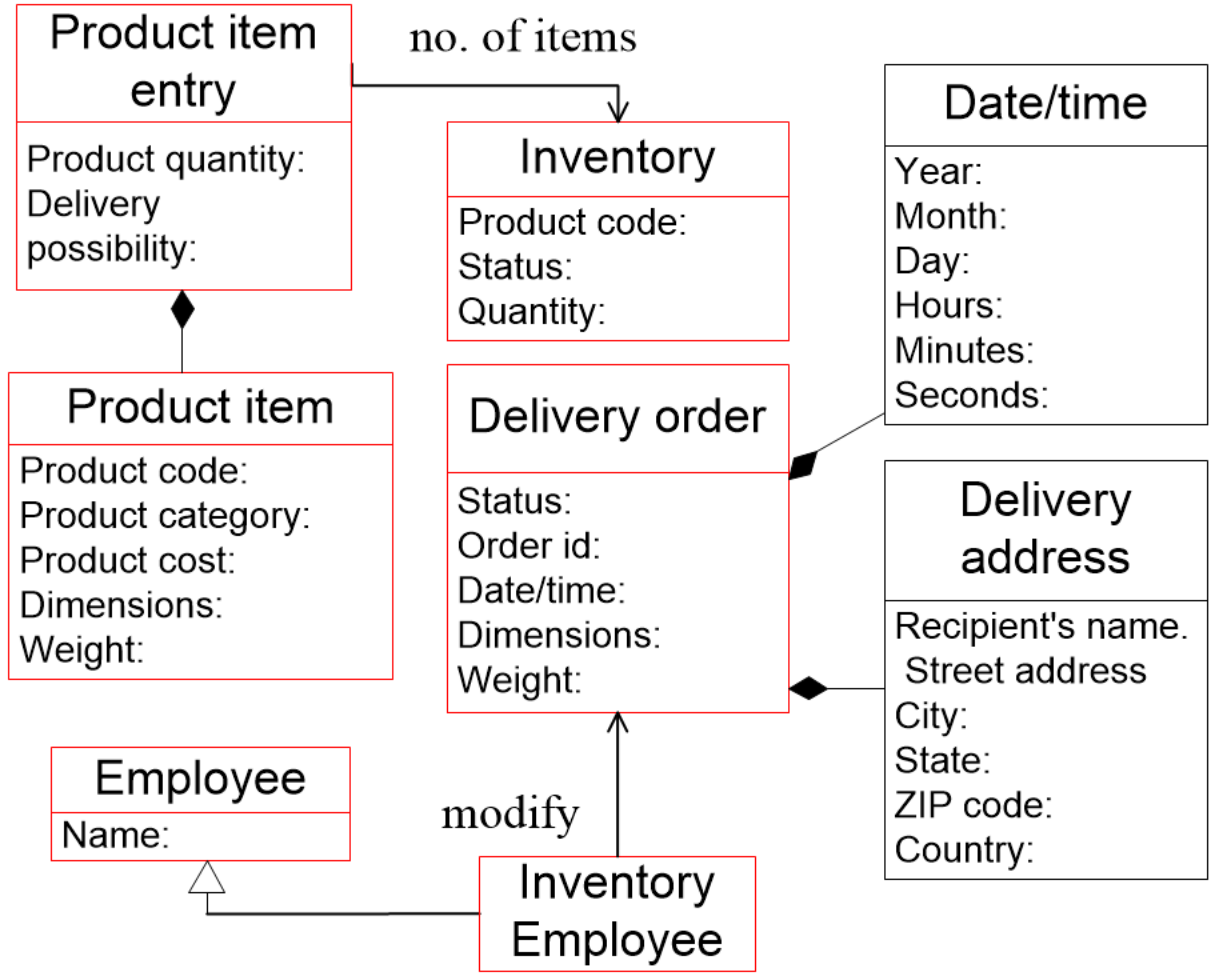
An entity do have distinctive identity.



DDD - Tactic design

An entity is an object that is identified by its consistent thread of continuity, as opposed to traditional objects, which are defined by their attributes.

An entity do have distinctive identity.

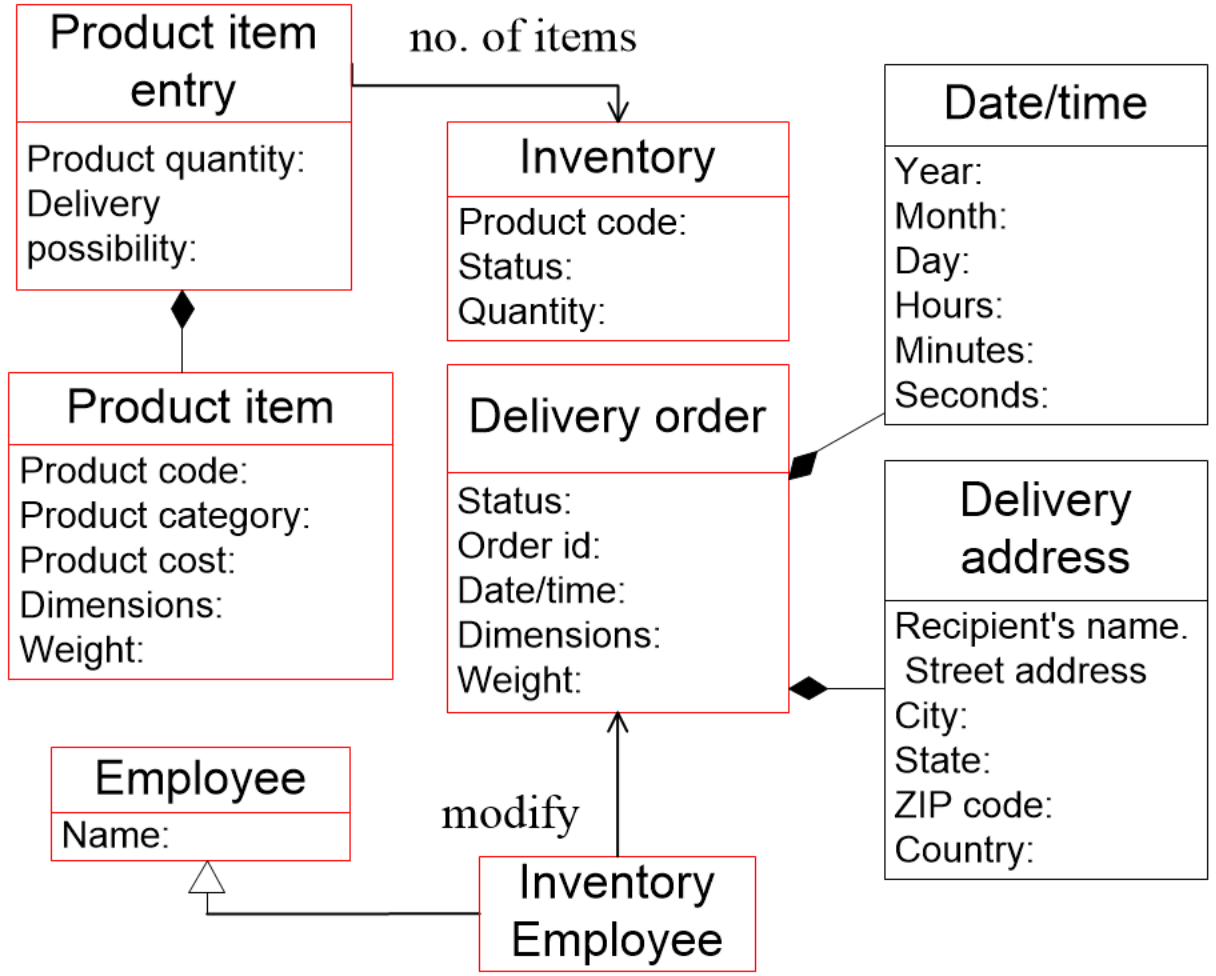


DDD - Tactic design

Value Objects are used to describe certain aspects of a domain.

Value Objects do not have distinct identity.

E.g., measures, quantity or describe the things in domain

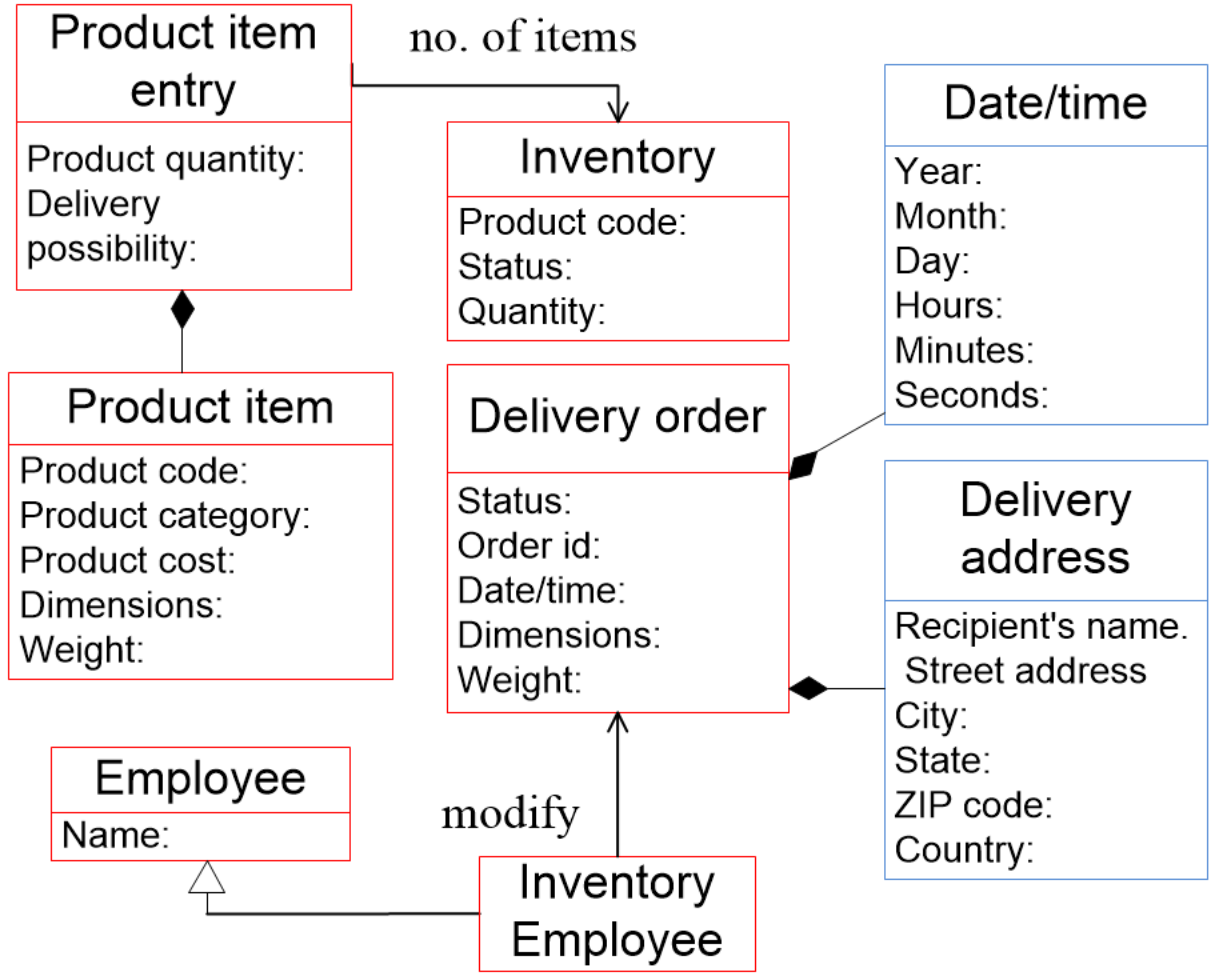


DDD - Tactic design

Value Objects are used to describe certain aspects of a domain.

Value Objects do not have distinct identity.

E.g., measures, quantity or describe the things in domain

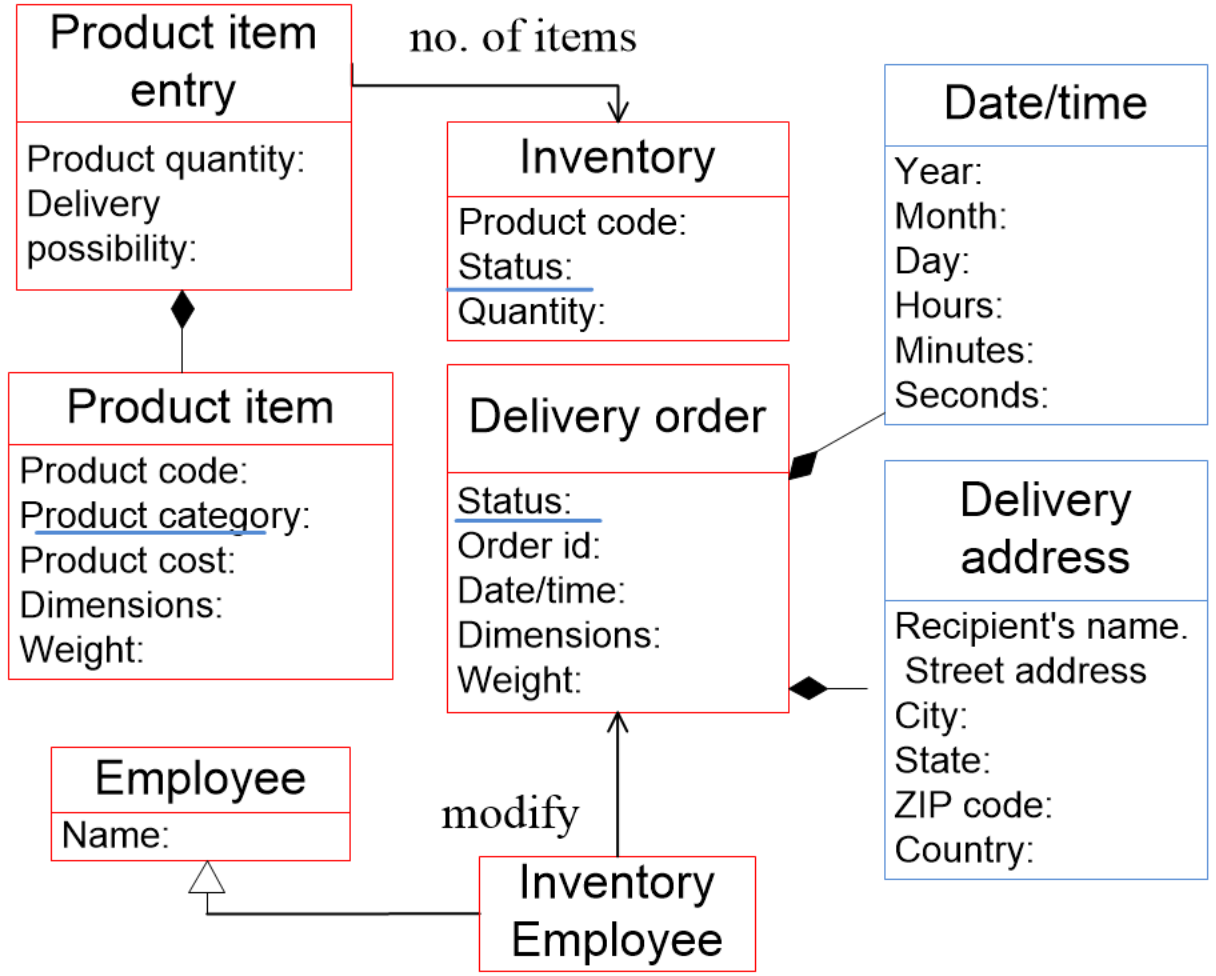


DDD - Tactic design

Value Objects are used to describe certain aspects of a domain.

Value Objects do not have distinct identity.

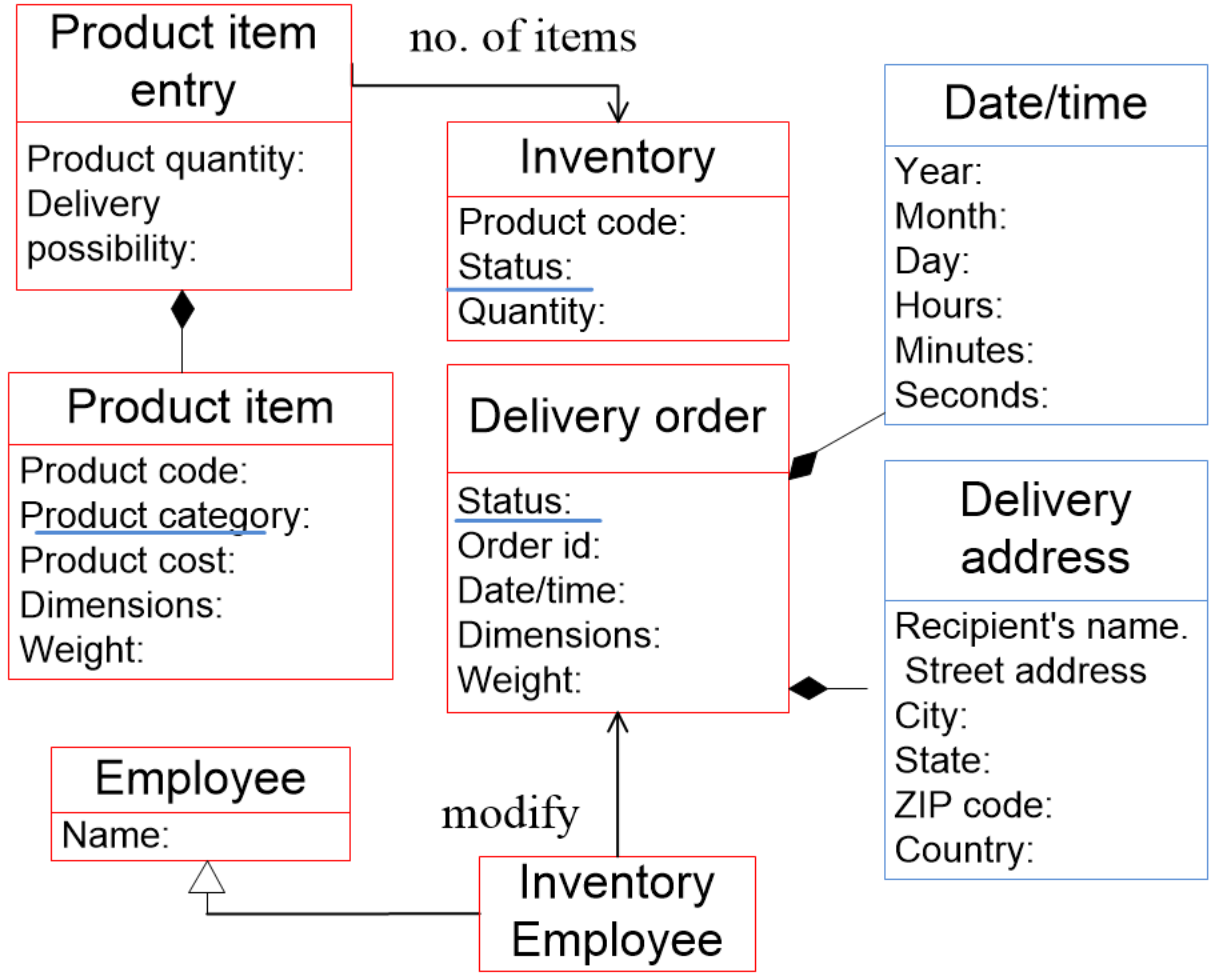
E.g., measures, quantity or describe the things in domain



DDD - Tactic design

An aggregate: a cluster of entities and value objects with a defined boundary.

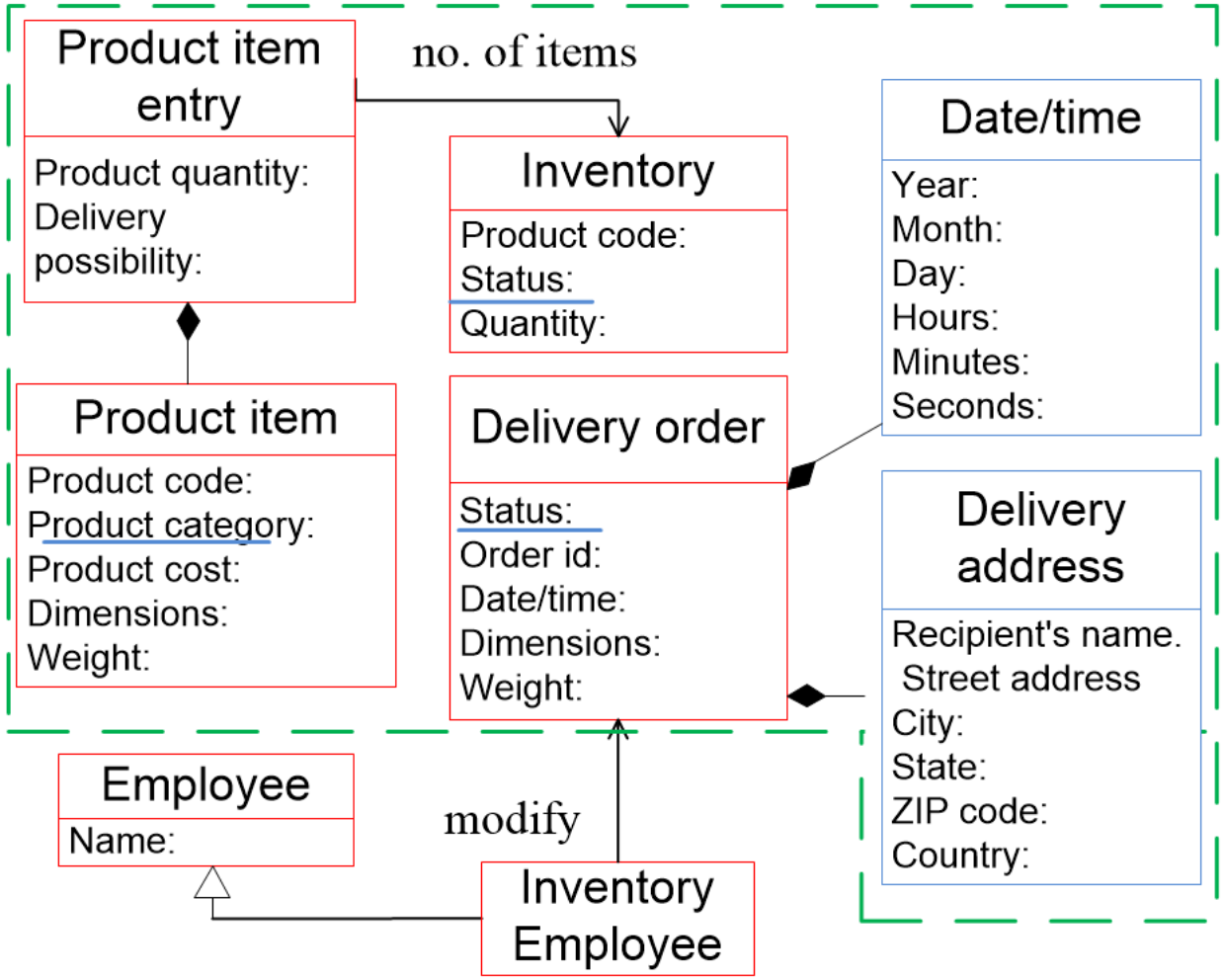
An aggregate has one root entity that is known as the ***aggregate root***.



DDD - Tactic design

An aggregate: a cluster of entities and value objects with a defined boundary.

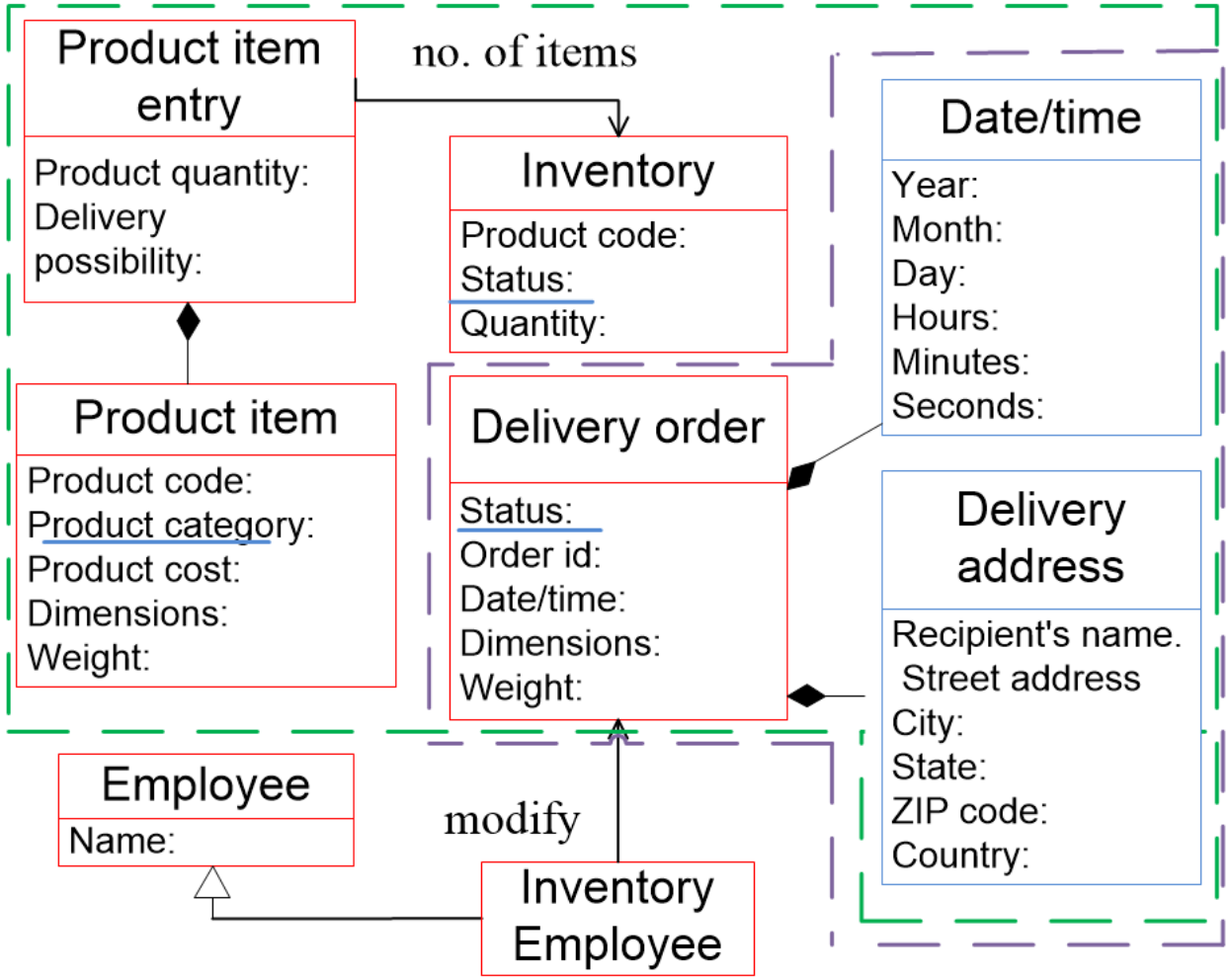
An aggregate has one root entity that is known as the ***aggregate root***.



DDD - Tactic design

An aggregate: a cluster of entities and value objects with a defined boundary.

An aggregate has one root entity that is known as the ***aggregate root***.



DDD - Tactic design

An aggregate: a cluster of entities and value objects with a defined boundary.

An aggregate has one root entity that is known as the ***aggregate root***.

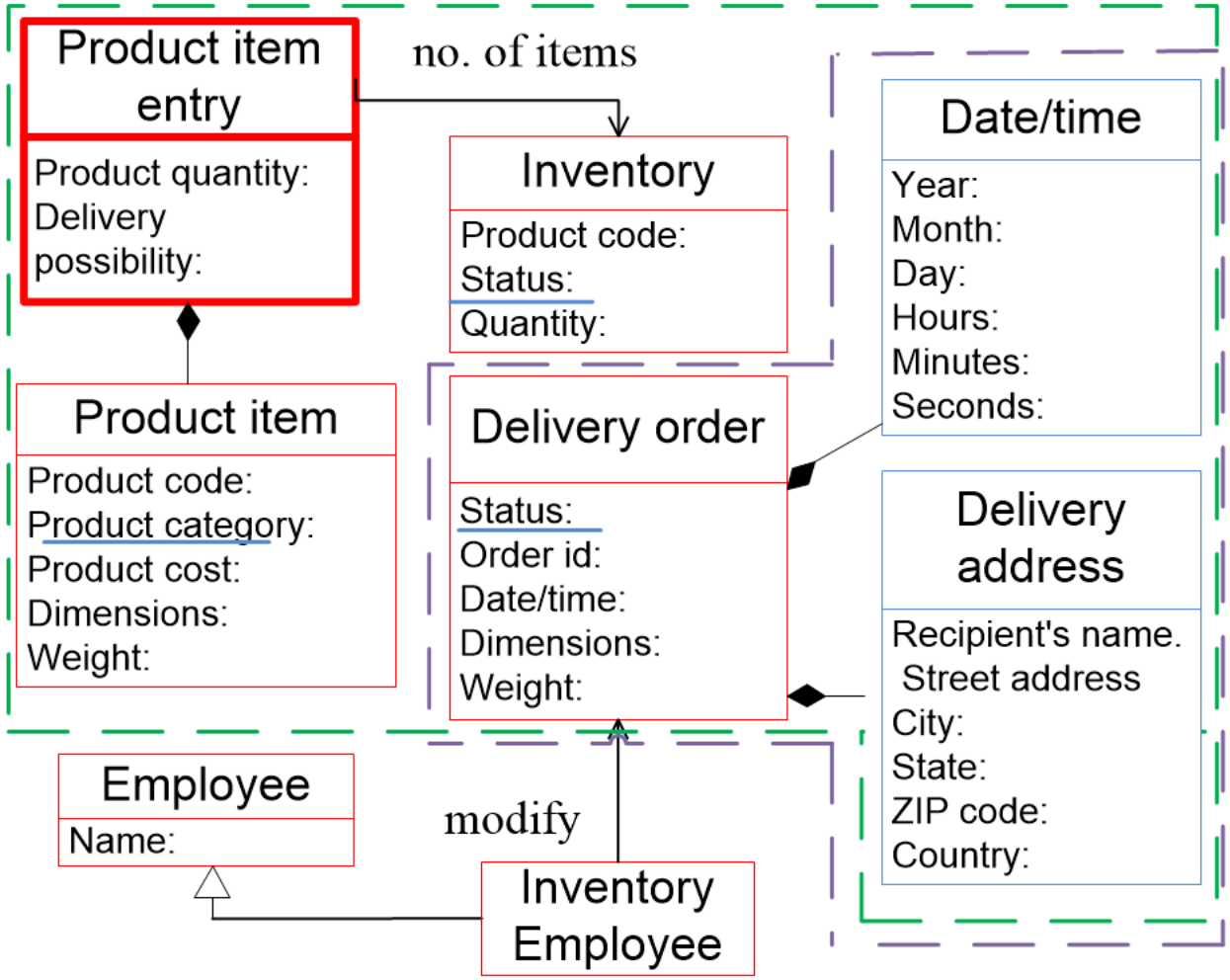
External objects have access only to the ***aggregate root***, and use that to pass along instructions to aggregate's enteritis.

DDD - Tactic design

An aggregate: a cluster of entities and value objects with a defined boundary.

An aggregate has one root entity that is known as the ***aggregate root***.

External objects have access only to the ***aggregate root***, and use that to pass along instructions to aggregate's enteritis.

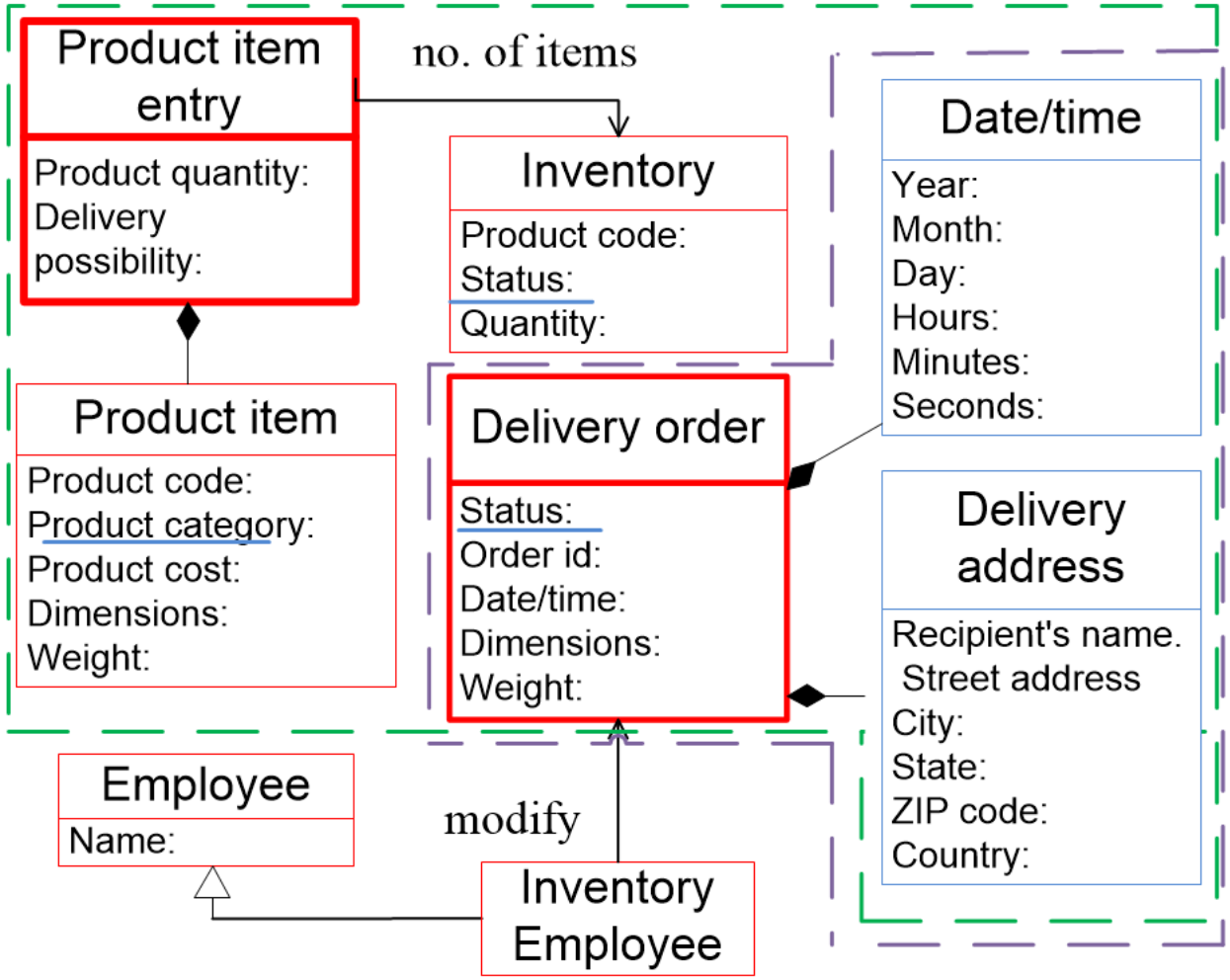


DDD - Tactic design

An aggregate: a cluster of entities and value objects with a defined boundary.

An aggregate has one root entity that is known as the ***aggregate root***.

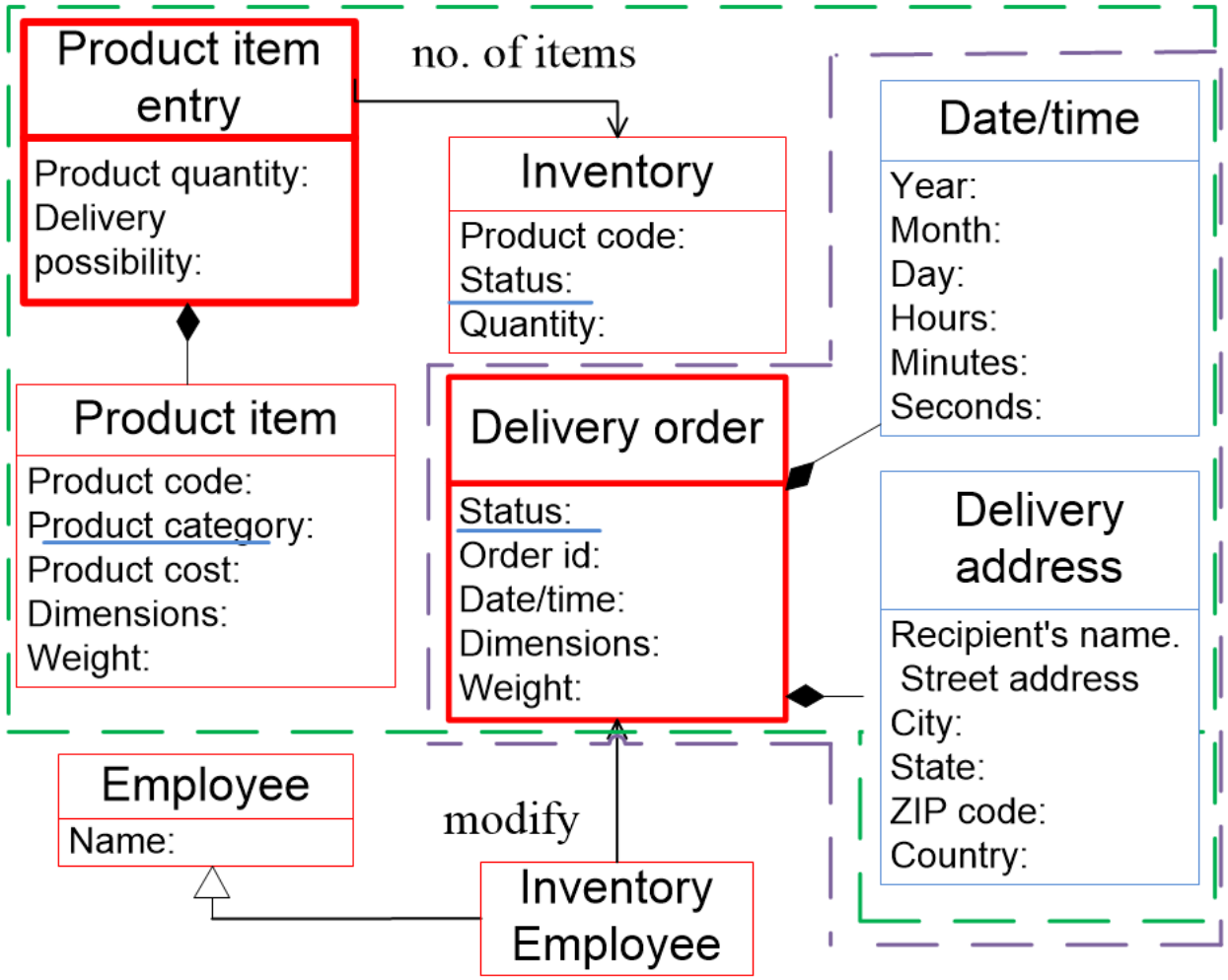
External objects have access only to the ***aggregate root***, and use that to pass along instructions to aggregate's enteritis.



DDD - Tactic design

A repository can be seen as an ***interface*** that is used to access to the data persistence system (the database).

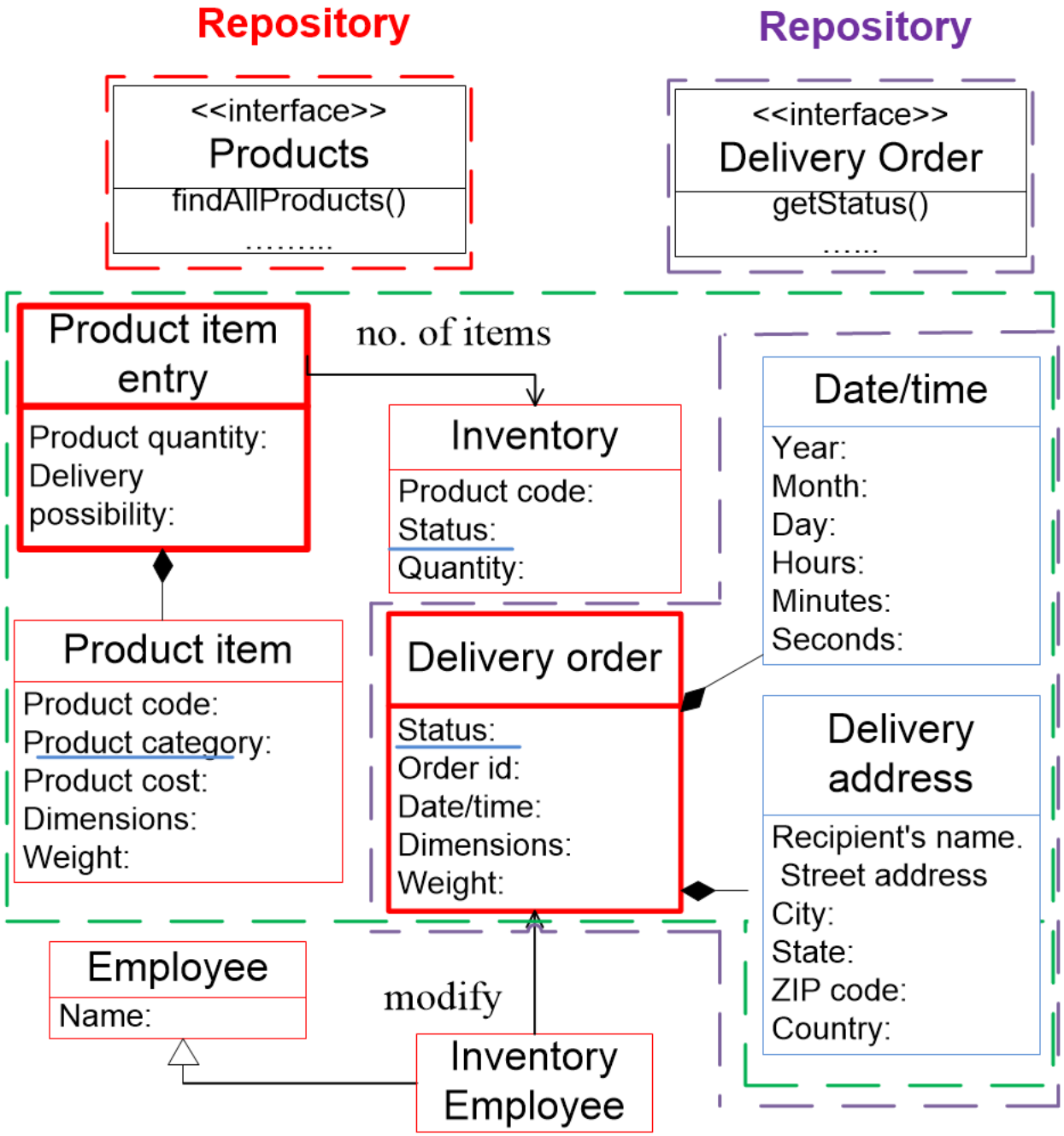
It is preferred to use a single repository per aggregate.



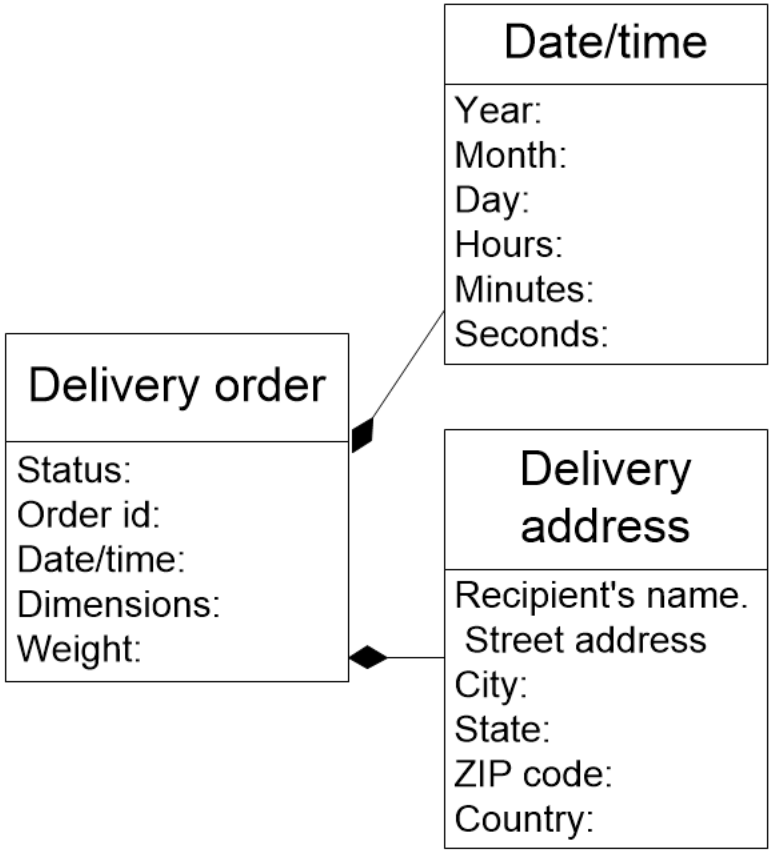
DDD - Tactic des

A repository can be seen as an ***interface*** that is used to access to the data persistence system (the database).

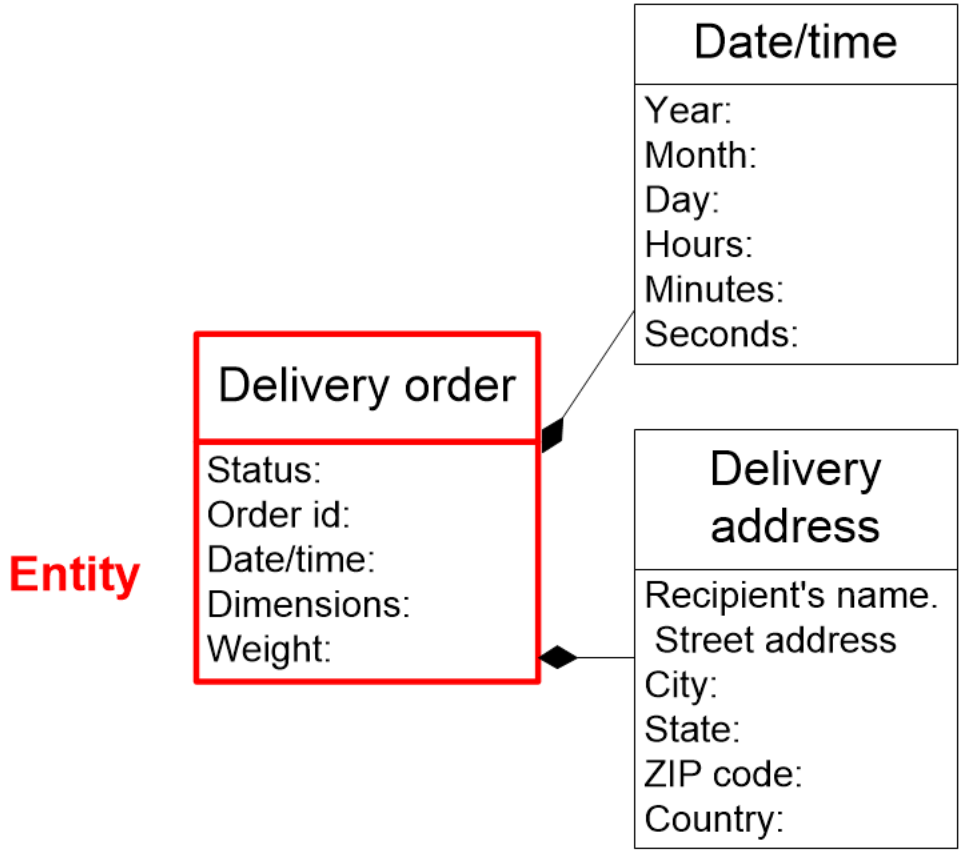
It is preferred to use a single repository per aggregate.



DDD - Tactic design – An example

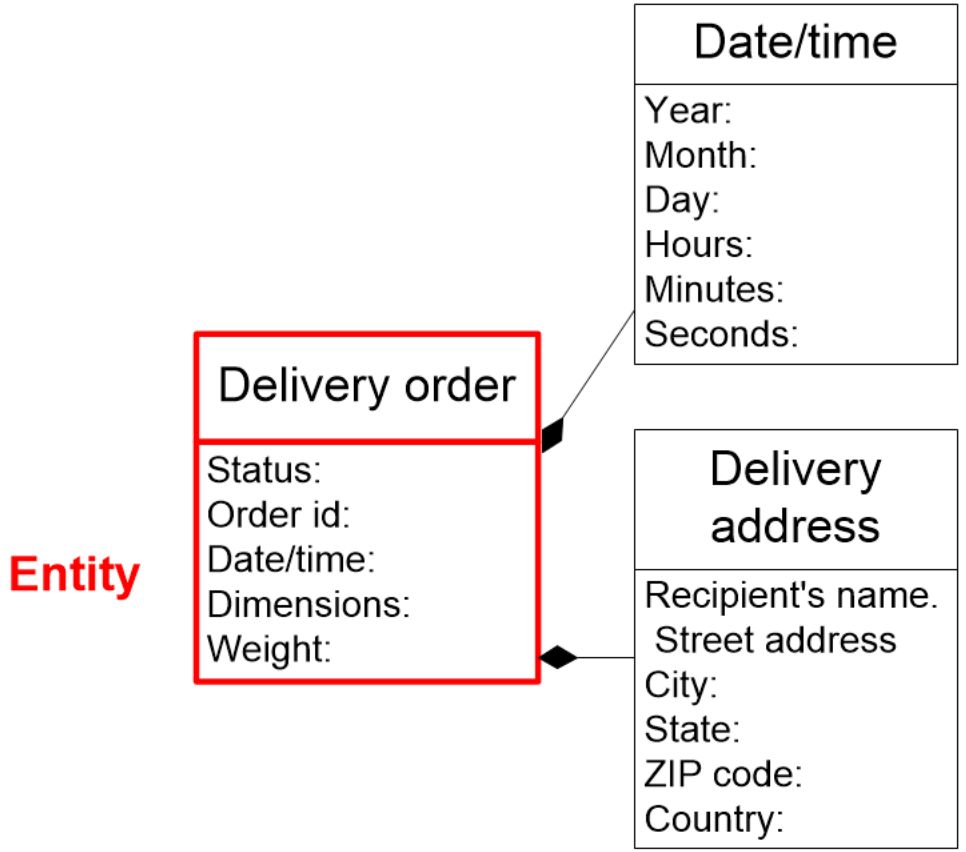


DDD - Tactic design – An example



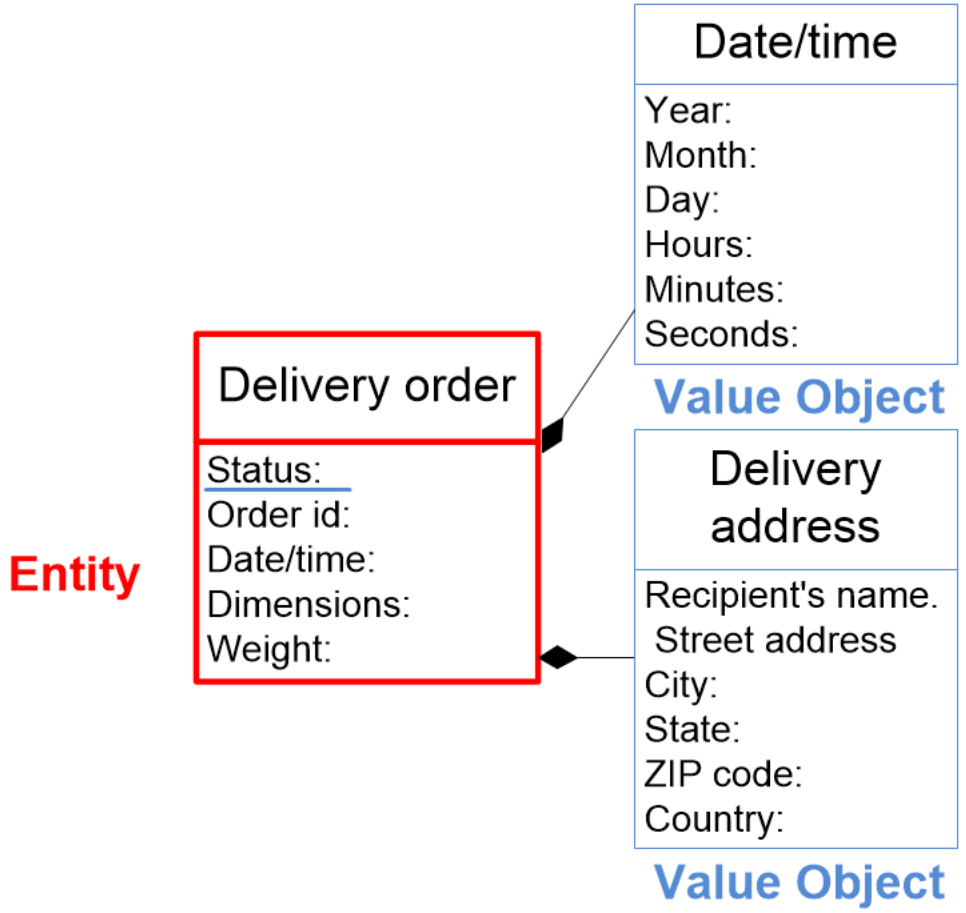
DDD - Tactic design – An example

```
//Entity
public class DeliveryOrder
  {
   OrderID: id
   ...
  }
```



DDD - Tactic design – An example

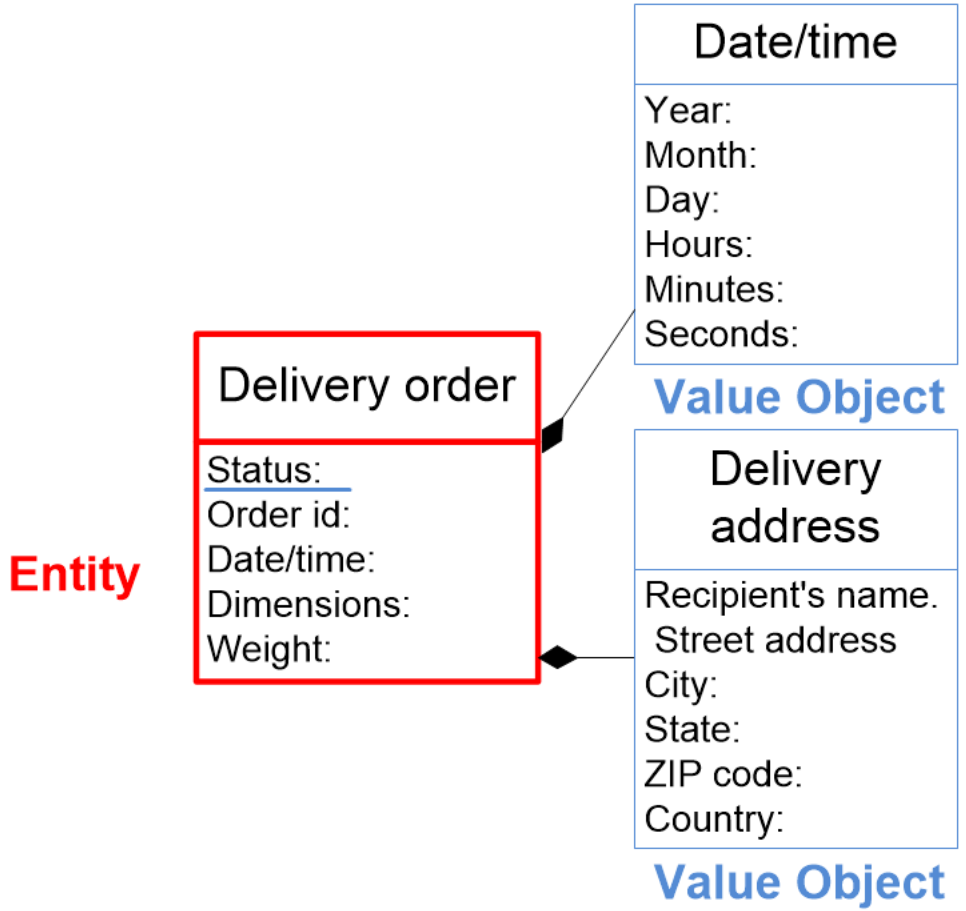
```
//Entity
public class DeliveryOrder
  {
   OrderID: id
   ...
  }
```



DDD - Tactic design – An example

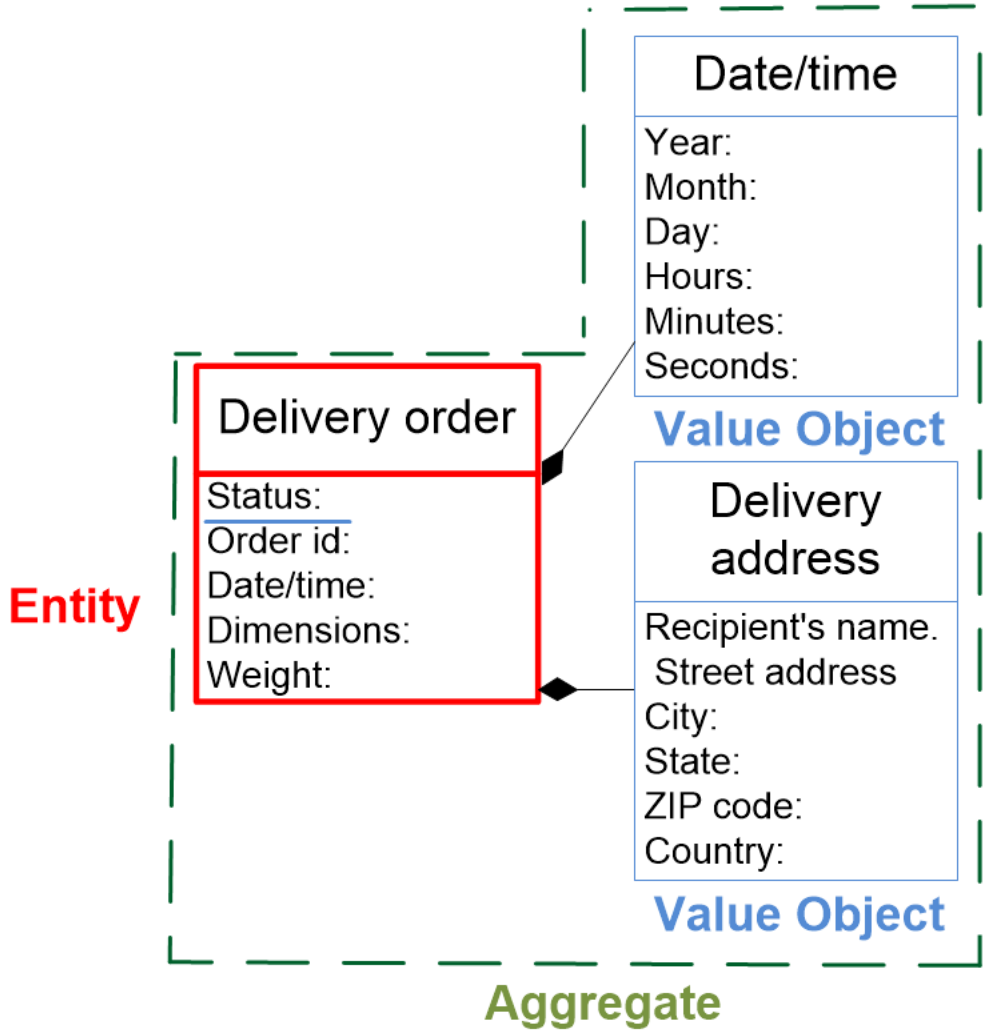
```
//Entity
public class DeliveryOrder
   {
    OrderID: id
    ...
   }
```

```
// Value Object
public class DateTime
{
    Year: year;
    Month: month;
    ...
}
// Value Object
public class DeliveryAddress
{
    RecName: string;
    StreetAddress : string
    ...
}
```



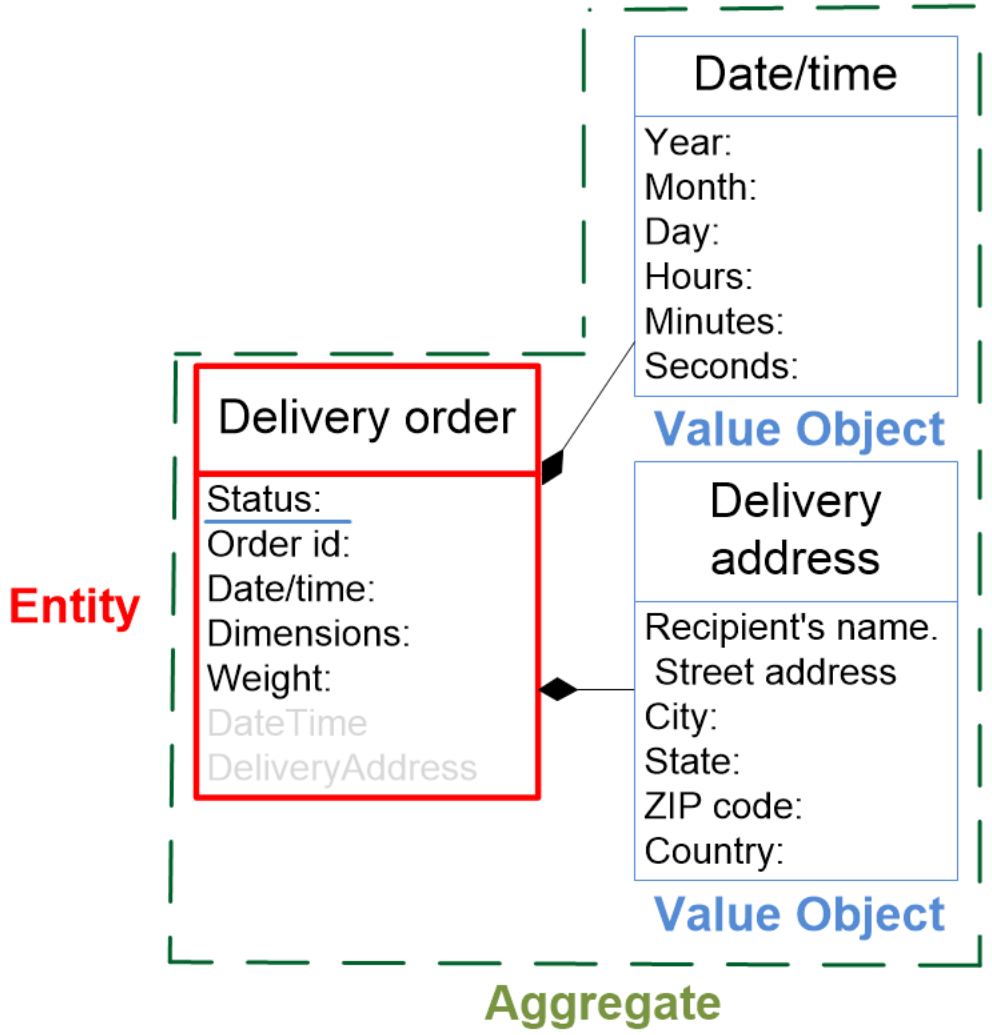
DDD - Tactic design – An example

```
//Entity
public class DeliveryOrder
   {
    OrderID: id
    ...
   }


// Value Object
public class DateTime
{
    Year: year;
    Month: month;
    …
}
// Value Object
public class DeliveryAddress
{
    RecName: string;
    StreetAddress : string
    …
}
```



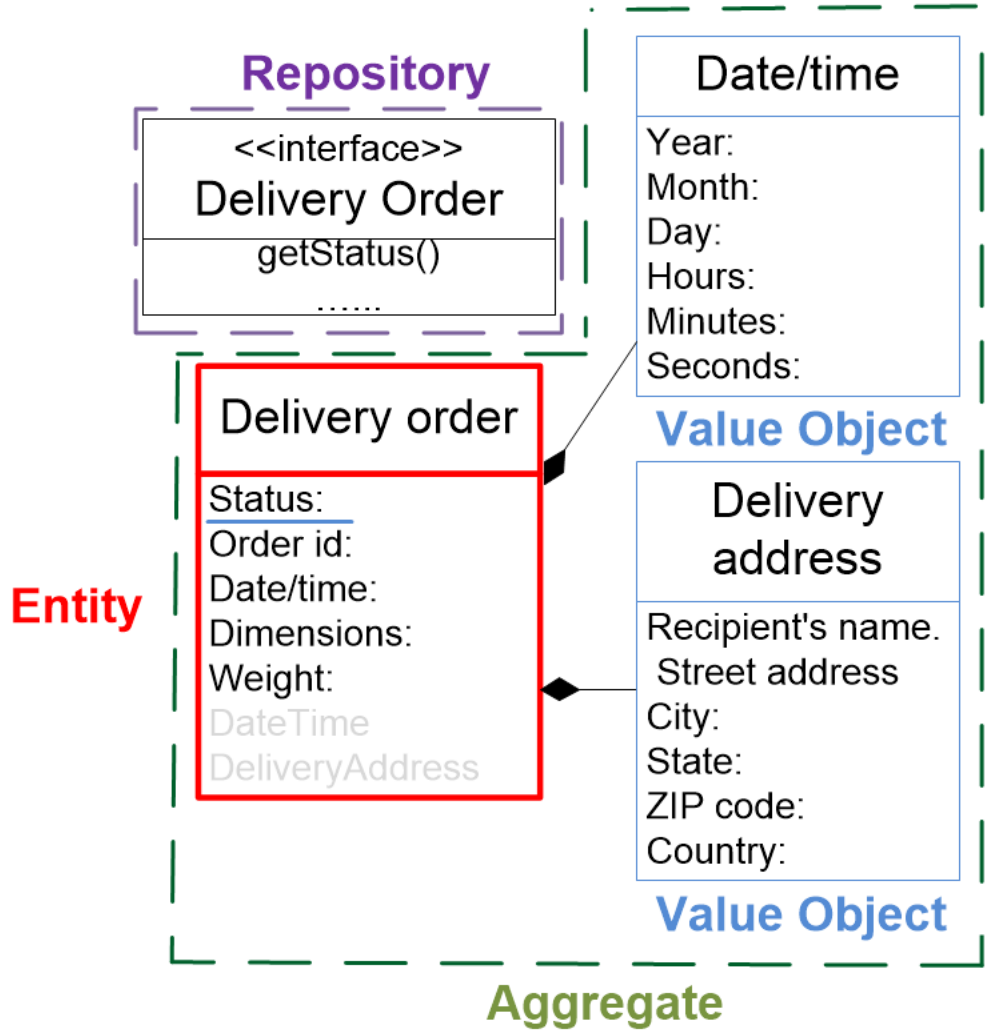
DDD - Tactic design – An example

```
//Entity
public class DeliveryOrder
   {
    OrderID: id
    ...
    DateTime:  dateTime;
    DeliveryAddress: deliveryAddress
   }

// Value Object
public class DateTime
{
    Year: year;
    Month: month;
    ...
}
// Value Object
public class DeliveryAddress
{
    RecName: string;
    StreetAddress : string
    ...
}
```



DDD - Tactic design – An example

```
//Entity
public class DeliveryOrder
   {
    OrderID: id
    ...
    DateTime:  dateTime;
    DeliveryAddress: deliveryAddress
   }

// Value Object
public class DateTime
{
    Year: year;
    Month: month;
    ...
}
// Value Object
public class DeliveryAddress
{
    RecName: string;
    StreetAddress : string
    ...
}
```



DDD for Microservices

DDD takes place in two phases:

- In the ***strategic phase*** we identify the BCs and map them out in a context map.
- In the ***tactical phase*** we model each BC according to the business rules of the subdomain.

Advantages of domain-driven design

Advantages of domain-driven design

1. The ubiquitous language ensures that the terminology and definitions that are used in the application are in line with activities and terminologies in the “real world”.
2. Facilitate communicating between development team and the domain experts. Accordingly, there is less chance of miscommunication or misunderstandings between them - **Business and development teams are aligned.**
3. Using the DDD approach guarantee that valuable knowledge is not lost when teams move on to other projects and when new people are brought in to maintain existing projects.
4. DDD offers great flexibility as the context definitions and boundaries make it easier to deal with requirement changes because everybody has the same understanding of the business domain.
5. Using the DDD approach leads to cleaner, more reliable code. It also can establish repeatable best practices and design patterns to make future projects run more smoothly and efficiently.

Disadvantages of domain-driven design

Disadvantages of domain-driven design

1. Requires extensive domain knowledge: You need to have at least one domain expert on the team. Without that kind of knowledge, you could end up with features that don’t fit into the domain context.
2. DDD is too complex for simple project. If the domain is relatively simple, DDD could be time-consuming overkill. If you do not need a dedicated domain expert, then you probably don’t need DDD.
3. Can be time consuming: Unless your developers are already domain experts, they will have to spend a lot of time with the business team to thoroughly understand the domain. But taking this time can eventually become an advantage when all of the domain knowledge is captured.
4. May not work well in highly technical projects because it is difficult to establish a common language that can be understood by people on the business side.

Additional resources

- Evans, Eric, and Eric J. Evans. **Domain-driven design: tackling complexity in the heart of software**. Addison-Wesley Professional, 2004.
- Millett, Scott, and Nick Tune. **Patterns, principles, and practices of domain-driven design**. John Wiley & Sons, 2015.
- Vernon, Vaughn. **Implementing domain-driven design**. Addison-Wesley, 2013.

UNIVERSITY OF TARTU

Thank You
for your attention

Mohamad Gharib
mohamad.gharib@ut.ee

unitartu

tartuuniversity

Enterprise System Integration
(MTAT.03.229)

ASSIGNMENT #2

MOHAMAD GHARIB

UNIVERSITY OF TARTU

Assignment #2

In your first assignment, you have defined a realistic scenario, **where the integration of 6-8 microservices is essential to deliver several functionalities of your system**. Also, you have defined the ***bounded contexts*** as well as the ***context maps*** (we called it back then a "***simple architecture***") of your integrated system.

In this assignment, you will demonstrate your ability in using the principles and patterns of the ***Domain-driven design (DDD) approach*** to develop a "**domain model**" for your integrated system.

Assignment #2

Task #1 (5 points): Develop a ***domain model*** of your ***integrated system,*** start by identifying ***key concepts*** of your ***system***, and their ***attributes***, then, define the ***relationships*** among the identified ***concepts***.

Task #2 (3 points): After finalizing your integrated domain model, use the ***four building blocks*** of the ***tactical design*** (***Entities, Value Objects, Aggregates, and Repositories***) to represent your ***domain model***.

Note: Represent any *repository* as an *interface*.

Rules for assignment #2

1. Your assignment should contain **two models**, one of them will be produced by **Task#1**, and the second will be produced by **Task#2**.
2. You need to submit your assignment as a **Word file** through **Moodle**, and the document should also contain your **Names**.
3. You have to submit your homework by the **defined deadline**, and you will **lose 0.5 point** for each hour of delay.
4. You can contact me **anytime, preferably using your team Slack channel**.
5. **30 March is the deadline to submit your assignment via Moodle, and 31 March** is the date to discuss this assignment.
6. **All team members should attend the assignment discussion**; you will not be allowed to discuss if your team is not complete. If the dedicated time slots for discussion do not fit your team, contact me and we can agree on a date/time that fits all your team members but within the same week of discussion.

UNIVERSITY OF TARTU

Thank You
for your attention

Mohamad Gharib
mohamad.gharib@ut.ee

unitartu

tartuuniversity